```
# Auto-generated by Django 4.2.11 on 2024-11-08 03:00

from django.db import migrations, models


class Migration(migrations.Migration):

    dependencies = [
        ("inventory", "0019_alter_product_sku"),
    ]

    operations = [
        migrations.AddField(
            model_name="product",
            name="weight_kg",
            field=models.DecimalField(blank=True, decimal_places=3, max_digits=8, null=True),
        ),
    ]
```

```
# Generated by Django 4.2.11 on 2024-11-08 03:00

from django.db import migrations, models


class Migration(migrations.Migration):

    dependencies = [
        ("inventory", "0020_product_weight_kg"),
    ]

    operations = [
        migrations.AddField(
            model_name="product",
            name="is_active",
            field=models.BooleanField(default=True),
        ),
    ]
```

```
# Generated by Django 4.2.11 on 2024-11-08 03:00

from django.db import migrations, models


class Migration(migrations.Migration):

    dependencies = [
        ("inventory", "0021_product_is_active"),
    ]

    operations = [
        migrations.AddField(
            model_name="product",
            name="barcode",
            field=models.CharField(blank=True, max_length=64, null=True),
        ),
    ]
```

```
# Generated by Django 4.2.11 on 2024-11-08 03:00

from django.db import migrations, models


class Migration(migrations.Migration):

    dependencies = [
        ("inventory", "0022_product_barcode"),
    ]

    operations = [
        migrations.AddField(
            model_name="product",
            name="reorder_point",
            field=models.PositiveIntegerField(default=0),
        ),
    ]
```

```
# Generated by Django 4.2.11 on 2024-11-08 03:00

from django.db import migrations, models


class Migration(migrations.Migration):

    dependencies = [
        ("inventory", "0023_product_reorder_point"),
    ]

    operations = [
        migrations.AddField(
            model_name="product",
            name="supplier_name",
            field=models.CharField(blank=True, max_length=128),
        ),
    ]
```

```
# Generated by Django 4.2.11 on 2024-11-08 03:00

from django.db import migrations, models


class Migration(migrations.Migration):

    dependencies = [
        ("inventory", "0024_product_supplier_name"),
    ]

    operations = [
        migrations.AddField(
            model_name="product",
            name="lead_time_days",
            field=models.PositiveIntegerField(default=7),
        ),
    ]
```

```
# Generated by Django 4.2.11 on 2024-11-08 03:00

from django.db import migrations, models


class Migration(migrations.Migration):

    dependencies = [
        ("inventory", "0025_product_lead_time_days"),
    ]

    operations = [
        migrations.AddField(
            model_name="product",
            name="unit_cost",
            field=models.DecimalField(blank=True, decimal_places=2, max_digits=10, null=True),
        ),
    ]
```

```
# Generated by Django 4.2.11 on 2024-11-08 03:00

from django.db import migrations, models


class Migration(migrations.Migration):

    dependencies = [
        ("inventory", "0026_product_unit_cost"),
    ]

    operations = [
        migrations.AddField(
            model_name="product",
            name="notes",
            field=models.TextField(blank=True),
        ),
    ]
```

```
# Generated by Django 4.2.11 on 2024-11-08 03:00

from django.db import migrations, models


class Migration(migrations.Migration):

    dependencies = [
        ("inventory", "0027_product_notes"),
    ]

    operations = [
        migrations.AddField(
            model_name="product",
            name="last_counted_at",
            field=models.DateTimeField(blank=True, null=True),
        ),
    ]
```

```
# Generated by Django 4.2.11 on 2024-11-08 03:00

from django.db import migrations, models


class Migration(migrations.Migration):

    dependencies = [
        ("inventory", "0028_product_last_counted_at"),
    ]

    operations = [
        migrations.AddField(
            model_name="product",
            name="is_perishable",
            field=models.BooleanField(default=False),
        ),
    ]
```

```
# Generated by Django 4.2.11 on 2024-11-08 03:00

from django.db import migrations, models


class Migration(migrations.Migration):

    dependencies = [
        ("inventory", "0029_product_is_perishable"),
    ]

    operations = [
        migrations.AddField(
            model_name="product",
            name="expiry_date",
            field=models.DateField(blank=True, null=True),
        ),
    ]
```

```
# Generated by Django 4.2.11 on 2024-11-08 03:00

from django.db import migrations, models


class Migration(migrations.Migration):

    dependencies = [
        ("inventory", "0030_product_expiry_date"),
    ]

    operations = [
        migrations.AddField(
            model_name="product",
            name="storage_location",
            field=models.CharField(blank=True, max_length=64),
        ),
    ]
```

```
# Generated by Django 4.2.11 on 2024-11-08 03:00

from django.db import migrations, models


class Migration(migrations.Migration):

    dependencies = [
        ("inventory", "0031_product_storage_location"),
    ]

    operations = [
        migrations.AddField(
            model_name="product",
            name="image_url",
            field=models.URLField(blank=True),
        ),
    ]
```

```
# Generated by Django 4.2.11 on 2024-11-08 03:00

from django.db import migrations, models


class Migration(migrations.Migration):

    dependencies = [
        ("inventory", "0032_product_image_url"),
    ]

    operations = [
        migrations.AddField(
            model_name="product",
            name="tax_rate",
            field=models.DecimalField(blank=True, decimal_places=4, max_digits=5, null=True),
        ),
    ]
```

```
# Generated by Django 4.2.11 on 2024-11-08 03:00

from django.db import migrations, models


class Migration(migrations.Migration):

    dependencies = [
        ("inventory", "0033_product_tax_rate"),
    ]

    operations = [
        migrations.AddField(
            model_name="product",
            name="is_discontinued",
            field=models.BooleanField(default=False),
        ),
    ]
```

```
# Generated by Django 4.2.11 on 2024-11-08 03:00

from django.db import migrations, models


class Migration(migrations.Migration):

    dependencies = [
        ("inventory", "0034_product_is_discontinued"),
    ]

    operations = [
        migrations.AddField(
            model_name="product",
            name="discontinued_at",
            field=models.DateTimeField(blank=True, null=True),
        ),
    ]
```

```
# Generated by Django 4.2.11 on 2024-11-08 03:00

from django.db import migrations, models


class Migration(migrations.Migration):

    dependencies = [
        ("inventory", "0035_product_discontinued_at"),
    ]

    operations = [
        migrations.AddField(
            model_name="product",
            name="replacement_sku",
            field=models.CharField(blank=True, max_length=64),
        ),
    ]
```

```
# Generated by Django 4.2.11 on 2024-11-08 03:00

from django.db import migrations, models


class Migration(migrations.Migration):

    dependencies = [
        ("inventory", "0036_product_replacement_sku"),
    ]

    operations = [
        migrations.AddField(
            model_name="product",
            name="min_order_qty",
            field=models.PositiveIntegerField(default=1),
        ),
    ]
```

```
# Generated by Django 4.2.11 on 2024-11-08 03:00

from django.db import migrations, models


class Migration(migrations.Migration):

    dependencies = [
        ("inventory", "0037_product_min_order_qty"),
    ]

    operations = [
        migrations.AddField(
            model_name="product",
            name="max_order_qty",
            field=models.PositiveIntegerField(blank=True, null=True),
        ),
    ]
```

```
# Generated by Django 4.2.11 on 2024-11-08 03:00

from django.db import migrations, models


class Migration(migrations.Migration):

    dependencies = [
        ("inventory", "0038_product_max_order_qty"),
    ]

    operations = [
        migrations.AddField(
            model_name="product",
            name="is_fragile",
            field=models.BooleanField(default=False),
        ),
    ]
```

```
# Generated by Django 4.2.11 on 2024-11-08 03:00

from django.db import migrations, models


class Migration(migrations.Migration):

    dependencies = [
        ("inventory", "0039_product_is_fragile"),
    ]

    operations = [
        migrations.AddField(
            model_name="product",
            name="requires_refrigeration",
            field=models.BooleanField(default=False),
        ),
    ]
```

```
# Generated by Django 4.2.11 on 2024-11-08 03:00

from django.db import migrations, models


class Migration(migrations.Migration):

    dependencies = [
        ("inventory", "0040_product_requires_refrigeration"),
    ]

    operations = [
        migrations.AddField(
            model_name="product",
            name="hazmat_class",
            field=models.CharField(blank=True, max_length=16),
        ),
    ]
```

```
# Generated by Django 4.2.11 on 2024-11-08 03:00

from django.db import migrations, models


class Migration(migrations.Migration):

    dependencies = [
        ("inventory", "0041_product_hazmat_class"),
    ]

    operations = [
        migrations.AddField(
            model_name="product",
            name="country_of_origin",
            field=models.CharField(blank=True, max_length=2),
        ),
    ]
```

```
# Generated by Django 4.2.11 on 2024-11-08 03:00

from django.db import migrations, models


class Migration(migrations.Migration):

    dependencies = [
        ("inventory", "0042_product_country_of_origin"),
    ]

    operations = [
        migrations.AddField(
            model_name="product",
            name="customs_code",
            field=models.CharField(blank=True, max_length=10),
        ),
    ]
```

```
# Generated by Django 4.2.11 on 2024-11-08 03:00

from django.db import migrations, models


class Migration(migrations.Migration):

    dependencies = [
        ("inventory", "0043_product_customs_code"),
    ]

    operations = [
        migrations.AddField(
            model_name="product",
            name="is_gift_wrappable",
            field=models.BooleanField(default=False),
        ),
    ]
```

```
# Generated by Django 4.2.11 on 2024-11-08 03:00

from django.db import migrations, models


class Migration(migrations.Migration):

    dependencies = [
        ("inventory", "0044_product_is_gift_wrappable"),
    ]

    operations = [
        migrations.AddField(
            model_name="product",
            name="gift_wrap_price",
            field=models.DecimalField(blank=True, decimal_places=2, max_digits=6, null=True),
        ),
    ]
```

```
# Generated by Django 4.2.11 on 2024-11-08 03:00

from django.db import migrations, models


class Migration(migrations.Migration):

    dependencies = [
        ("inventory", "0045_product_gift_wrap_price"),
    ]

    operations = [
        migrations.AddField(
            model_name="product",
            name="warranty_months",
            field=models.PositiveIntegerField(blank=True, null=True),
        ),
    ]
```

```
# Generated by Django 4.2.11 on 2024-11-08 03:00

from django.db import migrations, models


class Migration(migrations.Migration):

    dependencies = [
        ("inventory", "0046_product_warranty_months"),
    ]

    operations = [
        migrations.AddField(
            model_name="product",
            name="is_refurbished",
            field=models.BooleanField(default=False),
        ),
    ]
```

```
# Generated by Django 4.2.11 on 2024-11-08 03:00

from django.db import migrations, models


class Migration(migrations.Migration):

    dependencies = [
        ("inventory", "0047_product_is_refurbished"),
    ]

    operations = [
        migrations.AddField(
            model_name="product",
            name="refurbished_grade",
            field=models.CharField(blank=True, max_length=16),
        ),
    ]
```

```
# Generated by Django 4.2.11 on 2024-11-08 03:00

from django.db import migrations, models


class Migration(migrations.Migration):

    dependencies = [
        ("inventory", "0048_product_refurbished_grade"),
    ]

    operations = [
        migrations.AddField(
            model_name="product",
            name="is_bundle",
            field=models.BooleanField(default=False),
        ),
    ]
```

```
# Generated by Django 4.2.11 on 2024-11-08 03:00

from django.db import migrations, models


class Migration(migrations.Migration):

    dependencies = [
        ("inventory", "0049_product_is_bundle"),
    ]

    operations = [
        migrations.AddField(
            model_name="product",
            name="bundle_components",
            field=models.JSONField(blank=True, null=True),
        ),
    ]
```

```
# Generated by Django 4.2.11 on 2024-11-08 03:00

from django.db import migrations, models


class Migration(migrations.Migration):

    dependencies = [
        ("inventory", "0050_product_bundle_components"),
    ]

    operations = [
        migrations.AddField(
            model_name="product",
            name="is_digital",
            field=models.BooleanField(default=False),
        ),
    ]
```

```
# Generated by Django 4.2.11 on 2024-11-08 03:00

from django.db import migrations, models


class Migration(migrations.Migration):

    dependencies = [
        ("inventory", "0051_product_is_digital"),
    ]

    operations = [
        migrations.AddField(
            model_name="product",
            name="download_url",
            field=models.URLField(blank=True),
        ),
    ]
```

```
# Generated by Django 4.2.11 on 2024-11-08 03:00

from django.db import migrations, models


class Migration(migrations.Migration):

    dependencies = [
        ("inventory", "0052_product_download_url"),
    ]

    operations = [
        migrations.AddField(
            model_name="product",
            name="download_limit",
            field=models.PositiveIntegerField(blank=True, null=True),
        ),
    ]
```

```
# Generated by Django 4.2.11 on 2024-11-08 03:00

from django.db import migrations, models


class Migration(migrations.Migration):

    dependencies = [
        ("inventory", "0053_product_download_limit"),
    ]

    operations = [
        migrations.AddField(
            model_name="product",
            name="is_subscription",
            field=models.BooleanField(default=False),
        ),
    ]
```

```
# Generated by Django 4.2.11 on 2024-11-08 03:00

from django.db import migrations, models


class Migration(migrations.Migration):

    dependencies = [
        ("inventory", "0054_product_is_subscription"),
    ]

    operations = [
        migrations.AddField(
            model_name="product",
            name="subscription_interval",
            field=models.CharField(blank=True, max_length=16),
        ),
    ]
```

```
# Generated by Django 4.2.11 on 2024-11-08 03:00

from django.db import migrations, models


class Migration(migrations.Migration):

    dependencies = [
        ("inventory", "0055_product_subscription_interval"),
    ]

    operations = [
        migrations.AddField(
            model_name="product",
            name="subscription_price",
            field=models.DecimalField(blank=True, decimal_places=2, max_digits=6, null=True),
        ),
    ]
```

```
# Generated by Django 4.2.11 on 2024-11-08 03:00

from django.db import migrations, models


class Migration(migrations.Migration):

    dependencies = [
        ("inventory", "0056_product_subscription_price"),
    ]

    operations = [
        migrations.AddField(
            model_name="product",
            name="is_taxable",
            field=models.BooleanField(default=True),
        ),
    ]
```

```
# Generated by Django 4.2.11 on 2024-11-08 03:00

from django.db import migrations, models


class Migration(migrations.Migration):

    dependencies = [
        ("inventory", "0057_product_is_taxable"),
    ]

    operations = [
        migrations.AddField(
            model_name="product",
            name="tax_category",
            field=models.CharField(blank=True, max_length=32),
        ),
    ]
```

```
# Generated by Django 4.2.11 on 2024-11-08 03:00

from django.db import migrations, models


class Migration(migrations.Migration):

    dependencies = [
        ("inventory", "0058_product_tax_category"),
    ]

    operations = [
        migrations.AddField(
            model_name="product",
            name="is_clearance",
            field=models.BooleanField(default=False),
        ),
    ]
```

```
# Generated by Django 4.2.11 on 2024-11-08 03:00

from django.db import migrations, models


class Migration(migrations.Migration):

    dependencies = [
        ("inventory", "0059_product_is_clearance"),
    ]

    operations = [
        migrations.AddField(
            model_name="product",
            name="clearance_price",
            field=models.DecimalField(blank=True, decimal_places=2, max_digits=6, null=True),
        ),
    ]
```

```
# Generated by Django 4.2.11 on 2024-11-08 03:00

from django.db import migrations, models


class Migration(migrations.Migration):

    dependencies = [
        ("inventory", "0060_product_clearance_price"),
    ]

    operations = [
        migrations.AddField(
            model_name="product",
            name="is_preorder",
            field=models.BooleanField(default=False),
        ),
    ]
```

```
# Generated by Django 4.2.11 on 2024-11-08 03:00

from django.db import migrations, models


class Migration(migrations.Migration):

    dependencies = [
        ("inventory", "0061_product_is_preorder"),
    ]

    operations = [
        migrations.AddField(
            model_name="product",
            name="preorder_ship_date",
            field=models.DateField(blank=True, null=True),
        ),
    ]
```

```
# Generated by Django 4.2.11 on 2024-11-08 03:00

from django.db import migrations, models


class Migration(migrations.Migration):

    dependencies = [
        ("inventory", "0062_product_preorder_ship_date"),
    ]

    operations = [
        migrations.AddField(
            model_name="product",
            name="is_backorderable",
            field=models.BooleanField(default=False),
        ),
    ]
```

```
# Generated by Django 4.2.11 on 2024-11-08 03:00

from django.db import migrations, models


class Migration(migrations.Migration):

    dependencies = [
        ("inventory", "0063_product_is_backorderable"),
    ]

    operations = [
        migrations.AddField(
            model_name="product",
            name="backorder_limit",
            field=models.PositiveIntegerField(blank=True, null=True),
        ),
    ]
```

```
# Generated by Django 4.2.11 on 2024-11-08 03:00

from django.db import migrations, models


class Migration(migrations.Migration):

    dependencies = [
        ("inventory", "0064_product_backorder_limit"),
    ]

    operations = [
        migrations.AddField(
            model_name="product",
            name="is_featured",
            field=models.BooleanField(default=False),
        ),
    ]
```

```
# Generated by Django 4.2.11 on 2024-11-08 03:00

from django.db import migrations, models


class Migration(migrations.Migration):

    dependencies = [
        ("inventory", "0065_product_is_featured"),
    ]

    operations = [
        migrations.AddField(
            model_name="product",
            name="featured_rank",
            field=models.PositiveIntegerField(blank=True, null=True),
        ),
    ]
```

```
# Generated by Django 4.2.11 on 2024-11-08 03:00

from django.db import migrations, models


class Migration(migrations.Migration):

    dependencies = [
        ("inventory", "0066_product_featured_rank"),
    ]

    operations = [
        migrations.AddField(
            model_name="product",
            name="is_searchable",
            field=models.BooleanField(default=True),
        ),
    ]
```

```
# Generated by Django 4.2.11 on 2024-11-08 03:00

from django.db import migrations, models


class Migration(migrations.Migration):

    dependencies = [
        ("inventory", "0067_product_is_searchable"),
    ]

    operations = [
        migrations.AddField(
            model_name="product",
            name="search_keywords",
            field=models.TextField(blank=True),
        ),
    ]
```

```
# Generated by Django 4.2.11 on 2024-11-08 03:00

from django.db import migrations, models


class Migration(migrations.Migration):

    dependencies = [
        ("inventory", "0068_product_search_keywords"),
    ]

    operations = [
        migrations.AddField(
            model_name="product",
            name="meta_title",
            field=models.CharField(blank=True, max_length=70),
        ),
    ]
```

```
# Generated by Django 4.2.11 on 2024-11-08 03:00

from django.db import migrations, models


class Migration(migrations.Migration):

    dependencies = [
        ("inventory", "0069_product_meta_title"),
    ]

    operations = [
        migrations.AddField(
            model_name="product",
            name="meta_description",
            field=models.CharField(blank=True, max_length=160),
        ),
    ]
```

```
# Generated by Django 4.2.11 on 2024-11-08 03:00

from django.db import migrations, models


class Migration(migrations.Migration):

    dependencies = [
        ("inventory", "0070_product_meta_description"),
    ]

    operations = [
        migrations.AddField(
            model_name="product",
            name="canonical_url",
            field=models.URLField(blank=True),
        ),
    ]
```

```
# Generated by Django 4.2.11 on 2024-11-08 03:00

from django.db import migrations, models


class Migration(migrations.Migration):

    dependencies = [
        ("inventory", "0071_product_canonical_url"),
    ]

    operations = [
        migrations.AddField(
            model_name="product",
            name="is_noindex",
            field=models.BooleanField(default=False),
        ),
    ]
```

```
# Generated by Django 4.2.11 on 2024-11-08 03:00

from django.db import migrations, models


class Migration(migrations.Migration):

    dependencies = [
        ("inventory", "0072_product_is_noindex"),
    ]

    operations = [
        migrations.AddField(
            model_name="product",
            name="sitemap_priority",
            field=models.DecimalField(blank=True, decimal_places=1, max_digits=2, null=True),
        ),
    ]
```

```
# Generated by Django 4.2.11 on 2024-11-08 03:00

from django.db import migrations, models


class Migration(migrations.Migration):

    dependencies = [
        ("inventory", "0073_product_sitemap_priority"),
    ]

    operations = [
        migrations.AddField(
            model_name="product",
            name="sitemap_changefreq",
            field=models.CharField(blank=True, max_length=16),
        ),
    ]
```

```
# Generated by Django 4.2.11 on 2024-11-08 03:00

from django.db import migrations, models


class Migration(migrations.Migration):

    dependencies = [
        ("inventory", "0074_product_sitemap_changefreq"),
    ]

    operations = [
        migrations.AddField(
            model_name="product",
            name="is_archived",
            field=models.BooleanField(default=False),
        ),
    ]
```

```
# Generated by Django 4.2.11 on 2024-11-08 03:00

from django.db import migrations, models


class Migration(migrations.Migration):

    dependencies = [
        ("inventory", "0075_product_is_archived"),
    ]

    operations = [
        migrations.AddField(
            model_name="product",
            name="archived_at",
            field=models.DateTimeField(blank=True, null=True),
        ),
    ]
```

```
# Generated by Django 4.2.11 on 2024-11-08 03:00

from django.db import migrations, models


class Migration(migrations.Migration):

    dependencies = [
        ("inventory", "0076_product_archived_at"),
    ]

    operations = [
        migrations.AddField(
            model_name="product",
            name="archived_by",
            field=models.CharField(blank=True, max_length=64),
        ),
    ]
```

```
# Generated by Django 4.2.11 on 2024-11-08 03:00

from django.db import migrations, models


class Migration(migrations.Migration):

    dependencies = [
        ("inventory", "0077_product_archived_by"),
    ]

    operations = [
        migrations.AddField(
            model_name="product",
            name="is_locked",
            field=models.BooleanField(default=False),
        ),
    ]
```

```
# Generated by Django 4.2.11 on 2024-11-08 03:00

from django.db import migrations, models


class Migration(migrations.Migration):

    dependencies = [
        ("inventory", "0078_product_is_locked"),
    ]

    operations = [
        migrations.AddField(
            model_name="product",
            name="lock_reason",
            field=models.CharField(blank=True, max_length=128),
        ),
    ]
```

```
# Generated by Django 4.2.11 on 2024-11-08 03:00

from django.db import migrations, models


class Migration(migrations.Migration):

    dependencies = [
        ("inventory", "0079_product_lock_reason"),
    ]

    operations = [
        migrations.AddField(
            model_name="product",
            name="locked_by",
            field=models.CharField(blank=True, max_length=64),
        ),
    ]
```

```
# Generated by Django 4.2.11 on 2024-11-08 03:00

from django.db import migrations, models


class Migration(migrations.Migration):

    dependencies = [
        ("inventory", "0080_product_locked_by"),
    ]

    operations = [
        migrations.AddField(
            model_name="product",
            name="locked_at",
            field=models.DateTimeField(blank=True, null=True),
        ),
    ]
```

```
# Generated by Django 4.2.11 on 2024-11-08 03:00

from django.db import migrations, models


class Migration(migrations.Migration):

    dependencies = [
        ("inventory", "0081_product_locked_at"),
    ]

    operations = [
        migrations.AddField(
            model_name="product",
            name="is_hidden",
            field=models.BooleanField(default=False),
        ),
    ]
```

```
# Generated by Django 4.2.11 on 2024-11-08 03:00

from django.db import migrations, models


class Migration(migrations.Migration):

    dependencies = [
        ("inventory", "0082_product_is_hidden"),
    ]

    operations = [
        migrations.AddField(
            model_name="product",
            name="hidden_reason",
            field=models.CharField(blank=True, max_length=128),
        ),
    ]
```

```
# Generated by Django 4.2.11 on 2024-11-08 03:00

from django.db import migrations, models


class Migration(migrations.Migration):

    dependencies = [
        ("inventory", "0083_product_hidden_reason"),
    ]

    operations = [
        migrations.AddField(
            model_name="product",
            name="hidden_by",
            field=models.CharField(blank=True, max_length=64),
        ),
    ]
```

```
# Generated by Django 4.2.11 on 2024-11-08 03:00

from django.db import migrations, models


class Migration(migrations.Migration):

    dependencies = [
        ("inventory", "0084_product_hidden_by"),
    ]

    operations = [
        migrations.AddField(
            model_name="product",
            name="hidden_at",
            field=models.DateTimeField(blank=True, null=True),
        ),
    ]
```

```
# Generated by Django 4.2.11 on 2024-11-08 03:00

from django.db import migrations, models


class Migration(migrations.Migration):

    dependencies = [
        ("inventory", "0085_product_hidden_at"),
    ]

    operations = [
        migrations.AddField(
            model_name="product",
            name="is_deleted",
            field=models.BooleanField(default=False),
        ),
    ]
```

```
# Generated by Django 4.2.11 on 2024-11-08 03:00

from django.db import migrations, models


class Migration(migrations.Migration):

    dependencies = [
        ("inventory", "0086_product_is_deleted"),
    ]

    operations = [
        migrations.AddField(
            model_name="product",
            name="deleted_at",
            field=models.DateTimeField(blank=True, null=True),
        ),
    ]
```

```
# Generated by Django 4.2.11 on 2024-11-08 03:00

from django.db import migrations, models


class Migration(migrations.Migration):

    dependencies = [
        ("inventory", "0087_product_deleted_at"),
    ]

    operations = [
        migrations.AddField(
            model_name="product",
            name="deleted_by",
            field=models.CharField(blank=True, max_length=64),
        ),
    ]
```

```
# Generated by Django 4.2.11 on 2024-11-08 03:00

from django.db import migrations, models


class Migration(migrations.Migration):

    dependencies = [
        ("inventory", "0088_product_deleted_by"),
    ]

    operations = [
        migrations.AddField(
            model_name="product",
            name="is_merged",
            field=models.BooleanField(default=False),
        ),
    ]
```

```
# Generated by Django 4.2.11 on 2024-11-08 03:00

from django.db import migrations, models


class Migration(migrations.Migration):

    dependencies = [
        ("inventory", "0089_product_is_merged"),
    ]

    operations = [
        migrations.AddField(
            model_name="product",
            name="merged_into_sku",
            field=models.CharField(blank=True, max_length=64),
        ),
    ]
```

```
# Generated by Django 4.2.11 on 2024-11-08 03:00

from django.db import migrations, models


class Migration(migrations.Migration):

    dependencies = [
        ("inventory", "0090_product_merged_into_sku"),
    ]

    operations = [
        migrations.AddField(
            model_name="product",
            name="merged_at",
            field=models.DateTimeField(blank=True, null=True),
        ),
    ]
```

```
# Generated by Django 4.2.11 on 2024-11-08 03:00

from django.db import migrations, models


class Migration(migrations.Migration):

    dependencies = [
        ("inventory", "0091_product_merged_at"),
    ]

    operations = [
        migrations.AddField(
            model_name="product",
            name="is_split",
            field=models.BooleanField(default=False),
        ),
    ]
```

```
# Generated by Django 4.2.11 on 2024-11-08 03:00

from django.db import migrations, models


class Migration(migrations.Migration):

    dependencies = [
        ("inventory", "0092_product_is_split"),
    ]

    operations = [
        migrations.AddField(
            model_name="product",
            name="split_into_skus",
            field=models.JSONField(blank=True, null=True),
        ),
    ]
```

```
# Generated by Django 4.2.11 on 2024-11-08 03:00

from django.db import migrations, models


class Migration(migrations.Migration):

    dependencies = [
        ("inventory", "0093_product_split_into_skus"),
    ]

    operations = [
        migrations.AddField(
            model_name="product",
            name="split_at",
            field=models.DateTimeField(blank=True, null=True),
        ),
    ]
```

```
# Generated by Django 4.2.11 on 2024-11-08 03:00

from django.db import migrations, models


class Migration(migrations.Migration):

    dependencies = [
        ("inventory", "0094_product_split_at"),
    ]

    operations = [
        migrations.AddField(
            model_name="product",
            name="is_converted",
            field=models.BooleanField(default=False),
        ),
    ]
```

```
# Generated by Django 4.2.11 on 2024-11-08 03:00

from django.db import migrations, models


class Migration(migrations.Migration):

    dependencies = [
        ("inventory", "0095_product_is_converted"),
    ]

    operations = [
        migrations.AddField(
            model_name="product",
            name="converted_to_sku",
            field=models.CharField(blank=True, max_length=64),
        ),
    ]
```

```
# Generated by Django 4.2.11 on 2024-11-08 03:00

from django.db import migrations, models


class Migration(migrations.Migration):

    dependencies = [
        ("inventory", "0096_product_converted_to_sku"),
    ]

    operations = [
        migrations.AddField(
            model_name="product",
            name="converted_at",
            field=models.DateTimeField(blank=True, null=True),
        ),
    ]
```

```
# Generated by Django 4.2.11 on 2024-11-08 03:00

from django.db import migrations, models


class Migration(migrations.Migration):

    dependencies = [
        ("inventory", "0097_product_converted_at"),
    ]

    operations = [
        migrations.AddField(
            model_name="product",
            name="is_migrated",
            field=models.BooleanField(default=False),
        ),
    ]
```

```
# Generated by Django 4.2.11 on 2024-11-08 03:00

from django.db import migrations, models


class Migration(migrations.Migration):

    dependencies = [
        ("inventory", "0098_product_is_migrated"),
    ]

    operations = [
        migrations.AddField(
            model_name="product",
            name="migrated_from_system",
            field=models.CharField(blank=True, max_length=64),
        ),
    ]
```

```
# Generated by Django 4.2.11 on 2024-11-08 03:00

from django.db import migrations, models


class Migration(migrations.Migration):

    dependencies = [
        ("inventory", "0099_product_migrated_from_system"),
    ]

    operations = [
        migrations.AddField(
            model_name="product",
            name="migrated_at",
            field=models.DateTimeField(blank=True, null=True),
        ),
    ]
```

```
# Generated by Django 4.2.11 on 2024-11-08 03:00

from django.db import migrations, models


class Migration(migrations.Migration):

    dependencies = [
        ("inventory", "0100_product_migrated_at"),
    ]

    operations = [
        migrations.AddField(
            model_name="product",
            name="is_validated",
            field=models.BooleanField(default=False),
        ),
    ]
```

```
def _determine_keyword(
    cls, passed_keyword_args, mapping, values):
    api_version = get_api_version(
        cls._visual_studio_api_version_mapping,
        cls._minimum_api_version)
    kwargs = {}
    for arg_value, arg_type, key, default in mapping:
        if arg_value is None:
            value = default
        else:
            value = arg_value
        if value is not None:
            kwargs[key] = value
    return cls(**kwargs)

@classmethod
def from_pair_files(cls, private_key_file, cert_file,
                    passphrase=None):
    private_key = load_private_key_file(private_key_file,
                                        passphrase)
    cert = load_certificate_file(cert_file)
    return cls(private_key, cert)

@classmethod
def from_string_pem_serialized(cls, pem_bytes):
    cert = load_pem_certificate(pem_bytes)
    return cls(None, cert)

@classmethod
def from_private_key_der_data(cls, der_bytes,
                              passphrase=None):
    private_key = load_der_private_key(der_bytes,
                                       passphrase)
    return cls(private_key, None)

def public_key(self):
    return self._backend.x509_cert_public_key(self._x509_cert)

def subject_name_hash(self):
    return self._backend.x509_cert_subject_name_hash(
        self._x509_cert)

def __eq__(self, other):
    if not isinstance(other, Certificate):
        return NotImplemented

    return self._backend.x509_certs_equal(self._x509_cert,
                                          other._x509_cert)

def __ne__(self, other):
    return not self == other

def fingerprint(self, algorithm):
    return self._backend.x509_cert_fingerprint(self._x509_cert,
                                               algorithm)

def version(self):
    version = self._backend.x509_cert_version(self._x509_cert)
    if version == 0:
        return Version.v1
    else:
        raise UnknownVersion("Unknown X509 version: %d" % version)

def serial_number(self):
    return self._backend.x509_cert_serial_number(self._x509_cert)

def issuer_dn_bytes(self):
    return self._backend.x509_cert_issuer_dn(self._x509_cert)

def not_valid_before(self):
    return self._backend.x509_cert_not_valid_before(self._x509_cert)

def not_valid_after(self):
    return self._backend.x509_cert_not_valid_after(self._x509_cert)

def subject_dn_bytes(self):
    return self._backend.x509_cert_subject_dn(self._x509_cert)

def public_bytes(self, encoding):
    return self._backend.x509_cert_public_bytes(self._x509_cert,
                                                encoding)

def add_extension(self, extension, critical):
    if not isinstance(extension, ExtensionType):
        raise TypeError("extension must be an ExtensionType")

    extension = _RawExtension(extension.oid, critical,
                              extension.value_bytes())
    if any(ext.oid == extension.oid for ext in self._extensions):
        raise ValueError("This extension has already been set.")
    return self._replace(extensions=self._extensions + [extension])

def sign(self, private_key, algorithm):
    if self._subject_name is None:
        raise ValueError("A certificate must have a subject name")

    if self._issuer_name is None:
        raise ValueError("A certificate must have a issuer name")

    if self._serial_number is None:
        raise ValueError("A certificate must have a serial number")

    if self._not_valid_before is None:
        raise ValueError("A certificate must have a not valid before "
                         "time")

    if self._not_valid_after is None:
        raise ValueError("A certificate must have a not valid after "
                         "time")

    if self._public_key is None:
        raise ValueError("A certificate must have a public key")

    return self._backend.create_x509_certificate(self, private_key,
                                                 algorithm)

def __init__(self, backend, x509_cert):
    self._backend = backend
    self._x509_cert = x509_cert

def __repr__(self):
    return "<Certificate(subject={0}, ...)>".format(self.subject
```

```
```

```
# Auto-generated from news.json by tools/build_templates.py
# Do not edit manually

def render_item(item, variant=0):
    kicker = item["kicker"]
    headline = item["headline"]
    dek = item.get("dek", "")
    body = item["body"]
    author = item.get("author", "")
    time = item.get("time", "")

    if variant == 0:
        return f"""<article class="story">
  <p class="kicker">{kicker}</p>
  <h2>{headline}</h2>
  <p class="dek">{dek}</p>
  <p>{body}</p>
  <p class="author">{author} — {time}</p>
</article>"""
    elif variant == 1:
        return f"""<article class="story">
  <h2>{headline}</h2>
  <p>{body}</p>
</article>"""
    else:
        return f"""<article class="story">
  <p>{body}</p>
</article>"""

def render_items(items):
    return "\n".join(render_item(i, v % 3) for v, i in enumerate(items))
```

```
# ── Module-level constants ──────────────────────────────────────────────
MAX_RETRIES = 3
TIMEOUT = 30
DEBUG = False

class DataProcessor:
    """Processes data from various sources with caching support."""

    def __init__(self, source: str, cache_size: int = 100):
        self.source = source
        self.cache = {}
        self.cache_size = cache_size
        self._initialized = False

    def initialize(self) -> bool:
        """Initialize the processor and validate the source."""
        if self._initialized:
            return True
        print(f"Initializing processor for {self.source}")
        self._initialized = True
        return True

    def process(self, data: list) -> list:
        """Process a list of items, returning transformed results."""
        if not self._initialized:
            raise RuntimeError("Processor not initialized")
        results = []
        for item in data:
            if item in self.cache:
                results.append(self.cache[item])
            else:
                result = self._transform(item)
                self.cache[item] = result
                results.append(result)
        return results

    def _transform(self, item):
        return item.upper() if isinstance(item, str) else item

    def clear_cache(self):
        self.cache.clear()

def fetch_data(url: str, retries: int = MAX_RETRIES) -> dict:
    """Fetch data from a URL with retry logic."""
    for attempt in range(retries):
        try:
            print(f"Attempt {attempt + 1} for {url}")
            return {"status": "ok", "data": []}
        except Exception as e:
            if attempt == retries - 1:
                raise
    return {}

def main():
    processor = DataProcessor("example.com")
    processor.initialize()
    result = processor.process(["hello", "world"])
    print(result)

if __name__ == "__main__":
    main()
```

```
```

```
```

```
fn run_command_map() {
    let mut map = HashMap::new();
    map.insert("build", "cargo build");
    map.insert("test", "cargo test");
    map.insert("lint", "cargo clippy -- -D warnings");
    map.insert("fmt", "cargo fmt --check");
    map.insert("doc", "cargo doc --no-deps --open");
    map.insert("bench", "cargo bench");
    map.insert("clean", "cargo clean");
    map.insert("update", "cargo update");
    map.insert("audit", "cargo audit");
    map.insert("outdated", "cargo outdated");
    map.insert("tree", "cargo tree");
    map.insert("expand", "cargo expand");
    map.insert("watch", "cargo watch -x test");
    map.insert("release", "cargo build --release");
    map.insert("install", "cargo install --path .");
    map.insert("publish", "cargo publish --dry-run");
    map.insert("coverage", "cargo tarpaulin --out Html");
    map.insert("miri", "cargo +nightly miri test");
    map.insert("fuzz", "cargo fuzz run");
    map.insert("check", "cargo check --all-targets");
    map.insert("ci", "cargo fmt --check && cargo clippy -- -D warnings && cargo test");
    map.insert("docker", "docker build -t app .");
    map.insert("compose", "docker compose up -d");
    map.insert("logs", "docker compose logs -f");
    map.insert("migrate", "sqlx migrate run");
    map.insert("seed", "cargo run --bin seed");
    map.insert("serve", "cargo run --bin server");
    map.insert("worker", "cargo run --bin worker");
    map.insert("cron", "cargo run --bin cron");
    map.insert("docs", "mdbook build docs");
    map.insert("deploy", "./scripts/deploy.sh");
    map.insert("rollback", "./scripts/rollback.sh");
    map.insert("backup", "./scripts/backup.sh");
    map.insert("restore", "./scripts/restore.sh");
    map.insert("health", "curl -f http://localhost:8080/health");
    map.insert("metrics", "curl http://localhost:9090/metrics");
    map.insert("profile", "cargo flamegraph");
    map.insert("size", "cargo bloat --release");
    map.insert("deps", "cargo tree --duplicates");
    map.insert("unused", "cargo +nightly udeps");
    map.insert("msrv", "cargo msrv");
    map.insert("semver", "cargo semver-checks");
    map.insert("typos", "typos");
    map.insert("spell", "cargo spellcheck");
    map.insert("all", "cargo fmt --check && cargo clippy -- -D warnings && cargo test && cargo doc --no-deps");
    map
}

fn dispatch_command(name: &str) {
    let commands = run_command_map();
    match commands.get(name) {
        Some(cmd) => println!("Running: {}", cmd),
        None => println!("Unknown command: {}", name),
    }
}
```

```
fn build_query_with_joins(table: &str, joins: &[(&str, &str, &str)], filters: &[(&str, &str)]) -> String {
    let mut query = format!("SELECT * FROM {}", table);
    for (join_type, join_table, on_clause) in joins {
        query.push_str(&format!(" {} JOIN {} ON {}", join_type, join_table, on_clause));
    }
    if !filters.is_empty() {
        query.push_str(" WHERE ");
        let conditions: Vec<String> = filters.iter().map(|(col, val)| format!("{} = '{}'", col, val)).collect();
        query.push_str(&conditions.join(" AND "));
    }
    query
}

fn main() {
    let q = build_query_with_joins("users", &[("LEFT", "orders", "users.id = orders.user_id")], &[("users.active", "1")]);
    println!("{}", q);
}
```

```
use std::collections::HashMap;
use std::fmt;

#[derive(Debug, Clone, PartialEq)]
enum Value {
    Null,
    Bool(bool),
    Int(i64),
    Float(f64),
    Str(String),
    List(Vec<Value>),
    Map(HashMap<String, Value>),
}

impl fmt::Display for Value {
    fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result {
        match self {
            Value::Null => write!(f, "null"),
            Value::Bool(b) => write!(f, "{}", b),
            Value::Int(n) => write!(f, "{}", n),
            Value::Float(n) => write!(f, "{}", n),
            Value::Str(s) => write!(f, "\"{}\"", s),
            Value::List(items) => {
                let parts: Vec<String> = items.iter().map(|v| v.to_string()).collect();
                write!(f, "[{}]", parts.join(", "))
            }
            Value::Map(m) => {
                let parts: Vec<String> = m.iter().map(|(k, v)| format!("\"{}\": {}", k, v)).collect();
                write!(f, "{{{}}}", parts.join(", "))
            }
        }
    }
}

fn main() {
    let mut m = HashMap::new();
    m.insert("name".to_string(), Value::Str("Alice".to_string()));
    m.insert("age".to_string(), Value::Int(30));
    m.insert("active".to_string(), Value::Bool(true));
    let v = Value::Map(m);
    println!("{}", v);
}
```

```
use std::collections::HashMap;

fn word_count(text: &str) -> HashMap<&str, usize> {
    let mut counts = HashMap::new();
    for word in text.split_whitespace() {
        *counts.entry(word).or_insert(0) += 1;
    }
    counts
}

fn main() {
    let text = "the quick brown fox jumps over the lazy dog the fox";
    let counts = word_count(text);
    let mut pairs: Vec<_> = counts.iter().collect();
    pairs.sort_by(|a, b| b.1.cmp(a.1));
    for (word, count) in pairs {
        println!("{}: {}", word, count);
    }
```

```
fn main() -> Result<(), Box<dyn std::error::Error>> {
    let mut config = Config {
        host: "0.0.0.0".to_string(),
        port: 8080,
        workers: num_cpus::get(),
        max_connections: 25000,
        keep_alive: Duration::from_secs(75),
        request_timeout: Duration::from_secs(30),
        max_request_size: 10 * 1024 * 1024,
        enable_compression: true,
        compression_level: 6,
        enable_tls: false,
        tls_cert_path: None,
        tls_key_path: None,
    };

    config.load_from_env();

    println!("Starting server with {} workers", config.workers);

    let server = Server::new(config);
    server.run()?;

    Ok(())
}

struct Server {
    config: Config,
    router: Router,
    middleware: Vec<Box<dyn Middleware>>,
    shutdown_signal: Arc<AtomicBool>,
}

impl Server {
    fn new(config: Config) -> Self {
        Server {
            config,
            router: Router::new(),
            middleware: Vec::new(),
            shutdown_signal: Arc::new(AtomicBool::new(false)),
        }
    }

    fn add_middleware(&mut self, mw: Box<dyn Middleware>) {
        self.middleware.push(mw);
    }

    fn run(&self) -> Result<(), Box<dyn std::error::Error>> {
        let addr = format!("{}:{}", self.config.host, self.config.port);
        let listener = TcpListener::bind(&addr)?;
        listener.set_nonblocking(true)?;

        println!("Listening on {}", addr);

        let pool = ThreadPool::new(self.config.workers);

        while !self.shutdown_signal.load(Ordering::Relaxed) {
            match listener.accept() {
                Ok((stream, peer_addr)) => {
                    let router = self.router.clone();
                    let middleware = self.middleware.clone();
                    let config = self.config.clone();

                    pool.execute(move || {
                        if let Err(e) = handle_connection(stream, peer_addr, router, middleware, config) {
                            eprintln!("Connection error: {}", e);
                        }
                    });
                }
                Err(ref e) if e.kind() == std::io::ErrorKind::WouldBlock => {
                    std::thread::sleep(Duration::from_millis(10));
                }
                Err(e) => {
                    eprintln!("Accept error: {}", e);
                }
            }
        }

        println!("Shutting down server");
        Ok(())
    }

    fn shutdown(&self) {
        self.shutdown_signal.store(true, Ordering::Relaxed);
    }
}

fn handle_connection(
    mut stream: TcpStream,
    peer_addr: SocketAddr,
    router: Router,
    middleware: Vec<Box<dyn Middleware>>,
    config: Config,
) -> Result<(), Box<dyn std::error::Error>> {
    stream.set_read_timeout(Some(config.request_timeout))?;
    stream.set_write_timeout(Some(config.request_timeout))?;

    let mut buffer = vec![0; 8192];
    let bytes_read = stream.read(&mut buffer)?;

    if bytes_read == 0 {
        return Ok(());
    }

    let request = parse_request(&buffer[..bytes_read])?;

    let mut ctx = RequestContext {
        request,
        peer_addr,
        params: HashMap::new(),
        state: HashMap::new(),
    };

    let mut response = router.handle(&mut ctx, &middleware);

    if config.enable_compression {
        response.compress(config.compression_level);
    }

    stream.write_all(&response.to_bytes())?;
    stream.flush()?;

    Ok(())
}

struct RequestContext {
    request: Request,
    peer_addr: SocketAddr,
    params: HashMap<String, String>,
    state: HashMap<String, Box<dyn std::any::Any + Send>>,
}

impl RequestContext {
    fn get_param(&self, name: &str) -> Option<&str> {
        self.params.get(name).map(|s| s.as_str())
    }

    fn set_state<T: 'static + Send>(&mut self, key: &str, value: T) {
        self.state.insert(key.to_string(), Box::new(value));
    }

    fn get_state<T: 'static>(&self, key: &str) -> Option<&T> {
        self.state.get(key).and_then(|v| v.downcast_ref::<T>())
    }
}

struct Request {
    method: Method,
    path: String,
    query_string: String,
    version: HttpVersion,
    headers: HashMap<String, String>,
    body: Vec<u8>,
}

impl Request {
    fn header(&self, name: &str) -> Option<&str> {
        self.headers.get(&name.to_lowercase()).map(|s| s.as_str())
    }

    fn content_type(&self) -> Option<&str> {
        self.header("content-type")
    }

    fn content_length(&self) -> Option<usize> {
        self.header("content-length").and_then(|s| s.parse().ok())
    }

    fn is_keep_alive(&self) -> bool {
        match self.header("connection") {
            Some(v) if v.eq_ignore_ascii_case("close") => false,
            Some(v) if v.eq_ignore_ascii_case("keep-alive") => true,
            _ => self.version == HttpVersion::Http11,
        }
    }

    fn query_params(&self) -> HashMap<String, String> {
        parse_query_string(&self.query_string)
    }

    fn body_as_string(&self) -> Result<String, std::string::FromUtf8Error> {
        String::from_utf8(self.body.clone())
    }

    fn body_as_json<T: serde::de::DeserializeOwned>(&self) -> Result<T, serde_json::Error> {
        serde_json::from_slice(&self.body)
    }
}

#[derive(Debug, PartialEq, Clone, Copy)]
enum Method {
    Get,
    Post,
    Put,
    Delete,
    Patch,
    Head,
    Options,
    Trace,
    Connect,
}

impl Method {
    fn from_str(s: &str) -> Option<Method> {
        match s {
            "GET" => Some(Method::Get),
            "POST" => Some(Method::Post),
            "PUT" => Some(Method::Put),
            "DELETE" => Some(Method::Delete),
            "PATCH" => Some(Method::Patch),
            "HEAD" => Some(Method::Head),
            "OPTIONS" => Some(Method::Options),
            "TRACE" => Some(Method::Trace),
            "CONNECT" => Some(Method::Connect),
            _ => None,
        }
    }
}

#[derive(Debug, PartialEq, Clone, Copy)]
enum HttpVersion {
    Http10,
    Http11,
}

fn parse_request(data: &[u8]) -> Result<Request, Box<dyn std::error::Error>> {
    let mut lines = data.split(|&b| b == b'\n');

    let request_line = lines.next().ok_or("Empty request")?;
    let request_line = std::str::from_utf8(request_line)?.trim_end_matches('\r');
    let parts: Vec<&str> = request_line.splitn(3, ' ').collect();

    if parts.len() != 3 {
        return Err("Malformed request line".into());
    }

    let method = Method::from_str(parts[0]).ok_or("Unknown method")?;
    let (path, query_string) = match parts[1].find('?') {
        Some(pos) => (parts[1][..pos].to_string(), parts[1][pos + 1..].to_string()),
        None => (parts[1].to_string(), String::new()),
    };
    let version = match parts[2] {
        "HTTP/1.0" => HttpVersion::Http10,
        "HTTP/1.1" => HttpVersion::Http11,
        _ => return Err("Unsupported HTTP version".into()),
    };

    let mut headers = HashMap::new();
    for line in lines.by_ref() {
        let line = std::str::from_utf8(line)?.trim_end_matches('\r');
        if line.is_empty() {
            break;
        }
        if let Some(pos) = line.find(':') {
            let key = line[..pos].trim().to_lowercase();
            let value = line[pos + 1..].trim().to_string();
            headers.insert(key, value);
        }
    }

    let body: Vec<u8> = lines.flat_map(|l| l.iter().chain(std::iter::once(&b'\n'))).cloned().collect();

    Ok(Request {
        method,
        path,
        query_string,
        version,
        headers,
        body,
    })
}

fn parse_query_string(qs: &str) -> HashMap<String, String> {
    let mut params = HashMap::new();
    for pair in qs.split('&') {
        if let Some(pos) = pair.find('=') {
            let key = url_decode(&pair[..pos]);
            let value = url_decode(&pair[pos + 1..]);
            params.insert(key, value);
        }
    }
    params
}

fn url_decode(s: &str) -> String {
    let mut result = String::with_capacity(s.len());
    let mut chars = s.chars();
    while let Some(c) = chars.next() {
        match c {
            '%' => {
                let hex: String = chars.by_ref().take(2).collect();
                if let Ok(byte) = u8::from_str_radix(&hex, 16) {
                    result.push(byte as char);
                }
            }
            '+' => result.push(' '),
            _ => result.push(c),
        }
    }
    result
}

struct Response {
    status: StatusCode,
    headers: HashMap<String, String>,
    body: Vec<u8>,
}

impl Response {
    fn new(status: StatusCode) -> Self {
        Response {
            status,
            headers: HashMap::new(),
            body: Vec::new(),
        }
    }

    fn ok() -> Self {
        Response::new(StatusCode::Ok)
    }

    fn not_found() -> Self {
        Response::new(StatusCode::NotFound)
    }

    fn internal_error() -> Self {
        Response::new(StatusCode::InternalServerError)
    }

    fn bad_request() -> Self {
        Response::new(StatusCode::BadRequest)
    }

    fn set_header(&mut self, name: &str, value: &str) {
        self.headers.insert(name.to_string(), value.to_string());
    }

    fn with_header(mut self, name: &str, value: &str) -> Self {
        self.set_header(name, value);
        self
    }

    fn set_body(&mut self, body: Vec<u8>) {
        self.body = body;
    }

    fn with_body(mut self, body: Vec<u8>) -> Self {
        self.set_body(body);
        self
    }

    fn text(mut self, text: &str) -> Self {
        self.set_header("Content-Type", "text/plain; charset=utf-8");
        self.set_body(text.as_bytes().to_vec());
        self
    }

    fn html(mut self, html: &str) -> Self {
        self.set_header("Content-Type", "text/html; charset=utf-8");
        self.set_body(html.as_bytes().to_vec());
        self
    }

    fn json<T: serde::Serialize>(mut self, value: &T) -> Self {
        self.set_header("Content-Type", "application/json");
        self.set_body(serde_json::to_vec(value).unwrap_or_default());
        self
    }

    fn redirect(location: &str) -> Self {
        Response::new(StatusCode::MovedPermanently).with_header("Location", location)
    }

    fn compress(&mut self, level: u32) {
        if self.body.len() < 1024 {
            return;
        }
        let mut encoder = flate2::write::GzEncoder::new(Vec::new(), flate2::Compression::new(level));
        encoder.write_all(&self.body).unwrap();
        self.body = encoder.finish().unwrap();
        self.set_header("Content-Encoding", "gzip");
    }

    fn to_bytes(&self) -> Vec<u8> {
        let mut result = Vec::new();
        write!(result, "HTTP/1.1 {} {}\r\n", self.status.code(), self.status.reason()).unwrap();
        for (key, value) in &self.headers {
            write!(result, "{}: {}\r\n", key, value).unwrap();
        }
        write!(result, "Content-Length: {}\r\n", self.body.len()).unwrap();
        write!(result, "\r\n").unwrap();
        result.extend_from_slice(&self.body);
        result
    }
}

#[derive(Debug, Clone, Copy)]
enum StatusCode {
    Ok,
    Created,
    NoContent,
    MovedPermanently,
    Found,
    NotModified,
    BadRequest,
    Unauthorized,
    Forbidden,
    NotFound,
    MethodNotAllowed,
    InternalServerError,
    BadGateway,
    ServiceUnavailable,
}

impl StatusCode {
    fn code(&self) -> u16 {
        match self {
            StatusCode::Ok => 200,
            StatusCode::Created => 201,
            StatusCode::NoContent => 204,
            StatusCode::MovedPermanently => 301,
            StatusCode::Found => 302,
            StatusCode::NotModified => 304,
            StatusCode::BadRequest => 400,
            StatusCode::Unauthorized => 401,
            StatusCode::Forbidden => 403,
            StatusCode::NotFound => 404,
            StatusCode::MethodNotAllowed => 405,
            StatusCode::InternalServerError => 500,
            StatusCode::BadGateway => 502,
            StatusCode::ServiceUnavailable => 503,
        }
    }

    fn reason(&self) -> &str {
        match self {
            StatusCode::Ok => "OK",
            StatusCode::Created => "Created",
            StatusCode::NoContent => "No Content",
            StatusCode::MovedPermanently => "Moved Permanently",
            StatusCode::Found => "Found",
            StatusCode::NotModified => "Not Modified",
            StatusCode::BadRequest => "Bad Request",
            StatusCode::Unauthorized => "Unauthorized",
            StatusCode::Forbidden => "Forbidden",
            StatusCode::NotFound => "Not Found",
            StatusCode::MethodNotAllowed => "Method Not Allowed",
            StatusCode::InternalServerError => "Internal Server Error",
            StatusCode::BadGateway => "Bad Gateway",
            StatusCode::ServiceUnavailable => "Service Unavailable",
        }
    }
}

type Handler = Box<dyn Fn(&mut RequestContext) -> Response + Send + Sync>;

struct Route {
    method: Method,
    pattern: String,
    segments: Vec<PathSegment>,
    handler: Handler,
}

enum PathSegment {
    Literal(String),
    Param(String),
    Wildcard,
}

impl Route {
    fn new(method: Method, pattern: &str, handler: Handler) -> Self {
        let segments = pattern
            .trim_start_matches('/')
            .split('/')
            .map(|seg| {
                if seg == "*" {
                    PathSegment::Wildcard
                } else if seg.starts_with(':') {
                    PathSegment::Param(seg[1..].to_string())
                } else {
                    PathSegment::Literal(seg.to_string())
                }
            })
            .collect();

        Route {
            method,
            pattern: pattern.to_string(),
            segments,
            handler,
        }
    }

    fn matches(&self, method: Method, path: &str) -> Option<HashMap<String, String>> {
        if self.method != method {
            return None;
        }

        let path_segments: Vec<&str> = path.trim_start_matches('/').split('/').collect();
        let mut params = HashMap::new();

        for (i, segment) in self.segments.iter().enumerate() {
            match segment {
                PathSegment::Literal(lit) => {
                    if path_segments.get(i) != Some(&lit.as_str()) {
                        return None;
                    }
                }
                PathSegment::Param(name) => {
                    let value = path_segments.get(i)?;
                    params.insert(name.clone(), value.to_string());
                }
                PathSegment::Wildcard => {
                    let rest = path_segments[i..].join("/");
                    params.insert("*".to_string(), rest);
                    return Some(params);
                }
            }
        }

        if path_segments.len() == self.segments.len() {
            Some(params)
        } else {
            None
        }
    }
}

struct Router {
    routes: Vec<Route>,
    not_found_handler: Handler,
}

impl Router {
    fn new() -> Self {
        Router {
            routes: Vec::new(),
            not_found_handler: Box::new(|_| Response::not_found().text("Not Found")),
        }
    }

    fn add_route(&mut self, method: Method, pattern: &str, handler: Handler) {
        self.routes.push(Route::new(method, pattern, handler));
    }

    fn get(&mut self, pattern: &str, handler: Handler) {
        self.add_route(Method::Get, pattern, handler);
    }

    fn post(&mut self, pattern: &str, handler: Handler) {
        self.add_route(Method::Post, pattern, handler);
    }

    fn put(&mut self, pattern: &str, handler: Handler) {
        self.add_route(Method::Put, pattern, handler);
    }

    fn delete(&mut self, pattern: &str, handler: Handler) {
        self.add_route(Method::Delete, pattern, handler);
    }

    fn patch(&mut self, pattern: &str, handler: Handler) {
        self.add_route(Method::Patch, pattern, handler);
    }

    fn set_not_found(&mut self, handler: Handler) {
        self.not_found_handler = handler;
    }

    fn handle(&self, ctx: &mut RequestContext, middleware: &[Box<dyn Middleware>]) -> Response {
        let method = ctx.request.method;
        let path = ctx.request.path.clone();

        for route in &self.routes {
            if let Some(params) = route.matches(method, &path) {
                ctx.params = params;

                let mut response = None;
                for mw in middleware {
                    if let Some(r) = mw.before(ctx) {
                        response = Some(r);
                        break;
                    }
                }

                let mut response = response.unwrap_or_else(|| (route.handler)(ctx));

                for mw in middleware.iter().rev() {
                    mw.after(ctx, &mut response);
                }

                return response;
            }
        }

        (self.not_found_handler)(ctx)
    }
}

trait Middleware: Send + Sync {
    fn before(&self, ctx: &mut RequestContext) -> Option<Response>;
    fn after(&self, ctx: &RequestContext, response: &mut Response);
}

struct LoggingMiddleware;

impl Middleware for LoggingMiddleware {
    fn before(&self, ctx: &mut RequestContext) -> Option<Response> {
        println!(
            "{:?} {} from {}",
            ctx.request.method, ctx.request.path, ctx.peer_addr
        );
        None
    }

    fn after(&self, _ctx: &RequestContext, response: &mut Response) {
        println!("Response: {}", response.status.code());
    }
}

struct CorsMiddleware {
    allowed_origins: Vec<String>,
    allowed_methods: Vec<Method>,
    allowed_headers: Vec<String>,
    max_age: u32,
}

impl CorsMiddleware {
    fn new() -> Self {
        CorsMiddleware {
            allowed_origins: vec!["*".to_string()],
            allowed_methods: vec![
                Method::Get,
                Method::Post,
                Method::Put,
                Method::Delete,
                Method::Patch,
                Method::Options,
            ],
            allowed_headers: vec!["Content-Type".to_string(), "Authorization".to_string()],
            max_age: 86400,
        }
    }
}

impl Middleware for CorsMiddleware {
    fn before(&self, ctx: &mut RequestContext) -> Option<Response> {
        if ctx.request.method == Method::Options {
            let mut response = Response::new(StatusCode::NoContent);
            response.set_header("Access-Control-Allow-Origin", &self.allowed_origins.join(", "));
            response.set_header(
                "Access-Control-Allow-Methods",
                &self
                    .allowed_methods
                    .iter()
                    .map(|m| format!("{:?}", m).to_uppercase())
                    .collect::<Vec<_>>()
                    .join(", "),
            );
            response.set_header("Access-Control-Allow-Headers", &self.allowed_headers.join(", "));
            response.set_header("Access-Control-Max-Age", &self.max_age.to_string());
            return Some(response);
        }
        None
    }

    fn after(&self, _ctx: &RequestContext, response: &mut Response) {
        response.set_header("Access-Control-Allow-Origin", &self.allowed_origins.join(", "));
    }
}

struct RateLimitMiddleware {
    max_requests: u32,
    window: Duration,
    clients: Mutex<HashMap<String, (u32, Instant)>>,
}

impl RateLimitMiddleware {
    fn new(max_requests: u32, window: Duration) -> Self {
        RateLimitMiddleware {
            max_requests,
            window,
            clients: Mutex::new(HashMap::new()),
        }
    }
}

impl Middleware for RateLimitMiddleware {
    fn before(&self, ctx: &mut RequestContext) -> Option<Response> {
        let ip = ctx.peer_addr.ip().to_string();
        let mut clients = self.clients.lock().unwrap();
        let now = Instant::now();

        let entry = clients.entry(ip).or_insert((0, now));
        if now.duration_since(entry.1) > self.window {
            *entry = (1, now);
        } else {
            entry.0 += 1;
            if entry.0 > self.max_requests {
                return Some(
                    Response::new(StatusCode::ServiceUnavailable)
                        .text("Rate limit exceeded"),
                );
            }
        }
        None
    }

    fn after(&self, _ctx: &RequestContext, _response: &mut Response) {}
}

struct ThreadPool {
    workers: Vec<Worker>,
    sender: Option<std::sync::mpsc::Sender<Job>>,
}

type Job = Box<dyn FnOnce() + Send + 'static>;

struct Worker {
    id: usize,
    thread: Option<std::thread::JoinHandle<()>>,
}

impl ThreadPool {
    fn new(size: usize) -> ThreadPool {
        assert!(size > 0);

        let (sender, receiver) = std::sync::mpsc::channel();
        let receiver = Arc::new(Mutex::new(receiver));

        let mut workers = Vec::with_capacity(size);

        for id in 0..size {
            workers.push(Worker::new(id, Arc::clone(&receiver)));
        }

        ThreadPool {
            workers,
            sender: Some(sender),
        }
    }

    fn execute<F>(&self, f: F)
    where
        F: FnOnce() + Send + 'static,
    {
        let job = Box::new(f);
        self.sender.as_ref().unwrap().send(job).unwrap();
    }
}

impl Drop for ThreadPool {
    fn drop(&mut self) {
        drop(self.sender.take());

        for worker in &mut self.workers {
            if let Some(thread) = worker.thread.take() {
                thread.join().unwrap();
            }
        }
    }
}

impl Worker {
    fn new(id: usize, receiver: Arc<Mutex<std::sync::mpsc::Receiver<Job>>>) -> Worker {
        let thread = std::thread::spawn(move || loop {
            let job = receiver.lock().unwrap().recv();

            match job {
                Ok(job) => {
                    job();
                }
                Err(_) => {
                    break;
                }
            }
        });

        Worker {
            id,
            thread: Some(thread),
        }
    }
}

fn main() {
    let mut router = Router::new();

    router.get("/", Box::new(|_ctx| {
        Response::ok().text("Welcome to the server")
    }));

    router.get("/hello/:name", Box::new(|ctx| {
        let name = ctx.get_param("name").unwrap_or("world");
        Response::ok().text(&format!("Hello, {}!", name))
    }));

    router.get("/api/users", Box::new(|_ctx| {
        let users = vec!["alice", "bob", "charlie"];
        Response::ok().json(&users)
    }));

    router.post("/api/users", Box::new(|ctx| {
        match ctx.request.body_as_string() {
            Ok(body) => Response::new(StatusCode::Created).text(&format!("Created: {}", body)),
            Err(_) => Response::bad_request().text("Invalid body"),
        }
    }));

    router.get("/files/*", Box::new(|ctx| {
        let path = ctx.get_param("*").unwrap_or("");
        Response::ok().text(&format!("File path: {}", path))
    }));

    let mut server = Server::new(Config::default());
    server.router = router;
    server.add_middleware(Box::new(LoggingMiddleware));
    server.add_middleware(Box::new(CorsMiddleware::new()));
    server.add_middleware(Box::new(RateLimitMiddleware::new(100, Duration::from_secs(60))));

    server.run().unwrap();
}
```

```
# SuperMicro X11DPi-N motherboard configuration
# Dual Intel Xeon Scalable Processor Support
# 16 DIMM slots, up to 2TB DDR4-2933MHz ECC RDIMM
# Dual 10GbE LAN, IPMI 2.0 + KVM over LAN
# Intel C621 chipset, ATX form factor

# Boot order: NVMe first, then SATA, then network
boot_order = ["nvme", "sata", "network"]

# Fan control profiles
fan_profiles = {
  quiet = {
    cpu_fan_min_speed = 300
    cpu_fan_max_speed = 1200
    fan_curve = "linear"
  }
  performance = {
    cpu_fan_min_speed = 600
    cpu_fan_max_speed = 2400
    fan_curve = "aggressive"
  }
}

# Network configuration
network_config = {
  hostname = "supermicro-node01"
  domain = "datacenter.local"
  dns_servers = ["10.0.0.1", "10.0.0.2"]
  ntp_servers = ["pool.ntp.org"]
}

# IPMI configuration
ipmi_config = {
  ip_address = "10.0.1.100"
  netmask = "255.255.255.0"
  gateway = "10.0.1.1"
  vlan_id = 100
  user_name = "admin"
  user_password = "changeme123"
}

# Storage configuration
storage_config = {
  raid_controller = "software"
  raid_level = "raid10"
  hot_spare = true
  boot_drive = "/dev/nvme0n1"
  data_drives = ["/dev/nvme1n1", "/dev/nvme2n1", "/dev/nvme3n1", "/dev/nvme4n1"]
}

# Memory configuration
memory_config = {
  total_slots = 16
  populated_slots = 8
  memory_type = "DDR4-2933"
  ecc_enabled = true
  memory_mirroring = false
}

# CPU configuration
cpu_config = {
  socket_count = 2
  hyperthreading = true
  turbo_boost = true
  power_profile = "performance"
}

# Power management
power_config = {
  power_supply_redundancy = "1+1"
  power_limit = 1600
  power_recovery = "last_state"
  wake_on_lan = true
}

# Security settings
security_config = {
  tpm_enabled = true
  secure_boot = true
  intrusion_detection = true
  password_complexity = "high"
}

# Monitoring thresholds
monitoring_config = {
  cpu_temp_warning = 75
  cpu_temp_critical = 85
  system_temp_warning = 40
  system_temp_critical = 45
  fan_speed_warning = 500
  voltage_tolerance = 5
}

# Backup configuration
backup_config = {
  backup_server = "10.0.1.200"
  backup_protocol = "nfs"
  backup_path = "/backups/supermicro"
  backup_schedule = "0 2 * * *"
  retention_days = 30
}

# Update configuration
update_config = {
  bios_update_channel = "stable"
  firmware_update_channel = "stable"
  auto_update = false
  update_window = "maintenance"
}

# Logging configuration
logging_config = {
  log_level = "info"
  log_rotation = "daily"
  log_retention = 7
  remote_logging = true
  syslog_server = "10.0.1.201"
}

# Alert configuration
alert_config = {
  email_alerts = true
  email_recipients = ["admin@datacenter.local", "ops@datacenter.local"]
  snmp_traps = true
  snmp_community = "public"
  alert_threshold = "warning"
}

# Maintenance configuration
maintenance_config = {
  maintenance_window = "Sunday 02:00-06:00"
  auto_reboot = false
  reboot_delay = 300
  maintenance_mode = false
}

# Performance tuning
performance_config = {
  numa_enabled = true
  cpu_affinity = "auto"
  memory_interleaving = true
  prefetch_enabled = true
  turbo_boost = true
}

# Hardware monitoring
hardware_monitoring = {
  monitor_cpu = true
  monitor_memory = true
  monitor_storage = true
  monitor_network = true
  monitor_power = true
  monitor_fans = true
  monitor_temperatures = true
}

# Software configuration
software_config = {
  os_type = "linux"
  os_version = "ubuntu-22.04"
  kernel_version = "5.15.0-91-generic"
  driver_version = "latest"
  firmware_version = "latest"
}

# Network interfaces
network_interfaces = {
  eth0 = {
    type = "ethernet"
    speed = "10000"
    duplex = "full"
    mtu = 9000
    vlan = 100
    ip = "10.0.1.101"
    netmask = "255.255.255.0"
    gateway = "10.0.1.1"
    dns = ["10.0.0.1", "10.0.0.2"]
  }
  eth1 = {
    type = "ethernet"
    speed = "10000"
    duplex = "full"
    mtu = 9000
    vlan = 200
    ip = "10.0.2.101"
    netmask = "255.255.255.0"
    gateway = "10.0.2.1"
    dns = ["10.0.0.1", "10.0.0.2"]
  }
  ipmi = {
    type = "ethernet"
    speed = "1000"
    duplex = "full"
    mtu = 1500
    vlan = 100
    ip = "10.0.1.100"
    netmask = "255.255.255.0"
    gateway = "10.0.1.1"
    dns = ["10.0.0.1", "10.0.0.2"]
  }
}

# Storage devices
storage_devices = {
  nvme0n1 = {
    type = "nvme"
    size = "1TB"
    model = "Samsung 970 EVO Plus"
    serial = "S4EWNX0R123456"
    firmware = "2B2QEXM7"
    health = "good"
    temperature = 35
    power_on_hours = 1234
    power_cycles = 56
    unsafe_shutdowns = 2
    media_errors = 0
    error_log_entries = 0
    warning_temp_time = 0
    critical_temp_time = 0
    available_spare = 100
    available_spare_threshold = 10
    percentage_used = 5
    data_units_read = 1234567
    data_units_written = 2345678
    host_read_commands = 3456789
    host_write_commands = 4567890
    controller_busy_time = 5678
    power_cycles = 56
    power_on_hours = 1234
    unsafe_shutdowns = 2
    media_errors = 0
    error_log_entries = 0
    warning_temp_time = 0
    critical_temp_time = 0
    available_spare = 100
    available_spare_threshold = 10
    percentage_used = 5
    data_units_read = 1234567
    data_units_written = 2345678
    host_read_commands = 3456789
    host_write_commands = 4567890
    controller_busy_time = 5678
  }
  nvme1n1 = {
    type = "nvme"
    size = "1TB"
    model = "Samsung 970 EVO Plus"
    serial = "S4EWNX0R123457"
    firmware = "2B2QEXM7"
    health = "good"
    temperature = 36
    power_on_hours = 1234
    power_cycles = 56
    unsafe_shutdowns = 2
    media_errors = 0
    error_log_entries = 0
    warning_temp_time = 0
    critical_temp_time = 0
    available_spare = 100
    available_spare_threshold = 10
    percentage_used = 5
    data_units_read = 1234567
    data_units_written = 2345678
    host_read_commands = 3456789
    host_write_commands = 4567890
    controller_busy_time = 5678
  }
  nvme2n1 = {
    type = "nvme"
    size = "1TB"
    model = "Samsung 970 EVO Plus"
    serial = "S4EWNX0R123458"
    firmware = "2B2QEXM7"
    health = "good"
    temperature = 34
    power_on_hours = 1234
    power_cycles = 56
    unsafe_shutdowns = 2
    media_errors = 0
    error_log_entries = 0
    warning_temp_time = 0
    critical_temp_time = 0
    available_spare = 100
    available_spare_threshold = 10
    percentage_used = 5
    data_units_read = 1234567
    data_units_written = 2345678
    host_read_commands = 3456789
    host_write_commands = 4567890
    controller_busy_time = 5678
  }
  nvme3n1 = {
    type = "nvme"
    size = "1TB"
    model = "Samsung 970 EVO Plus"
    serial = "S4EWNX0R123459"
    firmware = "2B2QEXM7"
    health = "good"
    temperature = 35
    power_on_hours = 1234
    power_cycles = 56
    unsafe_shutdowns = 2
    media_errors = 0
    error_log_entries = 0
    warning_temp_time = 0
    critical_temp_time = 0
    available_spare = 100
    available_spare_threshold = 10
    percentage_used = 5
    data_units_read = 1234567
    data_units_written = 2345678
    host_read_commands = 3456789
    host_write_commands = 4567890
    controller_busy_time = 5678
  }
  nvme4n1 = {
    type = "nvme"
    size = "1TB"
    model = "Samsung 970 EVO Plus"
    serial = "S4EWNX0R123460"
    firmware = "2B2QEXM7"
    health = "good"
    temperature = 33
    power_on_hours = 1234
    power_cycles = 56
    unsafe_shutdowns = 2
    media_errors = 0
    error_log_entries = 0
    warning_temp_time = 0
    critical_temp_time = 0
    available_spare = 100
    available_spare_threshold = 10
    percentage_used = 5
    data_units_read = 1234567
    data_units_written = 2345678
    host_read_commands = 3456789
    host_write_commands = 4567890
    controller_busy_time = 5678
  }
}

# RAID configuration
raid_config = {
  raid_controller = "software"
  raid_level = "raid10"
  hot_spare = true
  boot_drive = "/dev/nvme0n1"
  data_drives = ["/dev/nvme1n1", "/dev/nvme2n1", "/dev/nvme3n1", "/dev/nvme4n1"]
  raid_status = "optimal"
  raid_health = "good"
  raid_temperature = 35
  raid_power_on_hours = 1234
  raid_power_cycles = 56
  raid_unsafe_shutdowns = 2
  raid_media_errors = 0
  raid_error_log_entries = 0
  raid_warning_temp_time = 0
  raid_critical_temp_time = 0
  raid_available_spare = 100
  raid_available_spare_threshold = 10
  raid_percentage_used = 5
  raid_data_units_read = 1234567
  raid_data_units_written = 2345678
  raid_host_read_commands = 3456789
  raid_host_write_commands = 4567890
  raid_controller_busy_time = 5678
}

# Memory modules
memory_modules = {
  dimm_a1 = {
    size = "32GB"
    type = "DDR4-2933"
    manufacturer = "Samsung"
    part_number = "M393A4K40CB2-CVF"
    serial = "12345678"
    ecc = true
    rank = 2
    speed = 2933
    voltage = 1.2
    temperature = 35
    error_count = 0
    correctable_errors = 0
    uncorrectable_errors = 0
    memory_health = "good"
    memory_status = "ok"
    memory_temperature = 35
    memory_power_on_hours = 1234
    memory_power_cycles = 56
    memory_unsafe_shutdowns = 2
    memory_media_errors = 0
    memory_error_log_entries = 0
    memory_warning_temp_time = 0
    memory_critical_temp_time = 0
    memory_available_spare = 100
    memory_available_spare_threshold = 10
    memory_percentage_used = 5
    memory_data_units_read = 1234567
    memory_data_units_written = 2345678
    memory_host_read_commands = 3456789
    memory_host_write_commands = 4567890
    memory_controller_busy_time = 5678
  }
  dimm_a2 = {
    size = "32GB"
    type = "DDR4-2933"
    manufacturer = "Samsung"
    part_number = "M393A4K40CB2-CVF"
    serial = "12345679"
    ecc = true
    rank = 2
    speed = 2933
    voltage = 1.2
    temperature = 36
    error_count = 0
    correctable_errors = 0
    uncorrectable_errors = 0
    memory_health = "good"
    memory_status = "ok"
    memory_temperature = 36
    memory_power_on_hours = 1234
    memory_power_cycles = 56
    memory_unsafe_shutdowns = 2
    memory_media_errors = 0
    memory_error_log_entries = 0
    memory_warning_temp_time = 0
    memory_critical_temp_time = 0
    memory_available_spare = 100
    memory_available_spare_threshold = 10
    memory_percentage_used = 5
    memory_data_units_read = 1234567
    memory_data_units_written = 2345678
    memory_host_read_commands = 3456789
    memory_host_write_commands = 4567890
    memory_controller_busy_time = 5678
  }
  dimm_b1 = {
    size = "32GB"
    type = "DDR4-2933"
    manufacturer = "Samsung"
    part_number = "M393A4K40CB2-CVF"
    serial = "12345680"
    ecc = true
    rank = 2
    speed = 2933
    voltage = 1.2
    temperature = 34
    error_count = 0
    correctable_errors = 0
    uncorrectable_errors = 0
    memory_health = "good"
    memory_status = "ok"
    memory_temperature = 34
    memory_power_on_hours = 1234
    memory_power_cycles = 56
    memory_unsafe_shutdowns = 2
    memory_media_errors = 0
    memory_error_log_entries = 0
    memory_warning_temp_time = 0
    memory_critical_temp_time = 0
    memory_available_spare = 100
    memory_available_spare_threshold = 10
    memory_percentage_used = 5
    memory_data_units_read = 1234567
    memory_data_units_written = 2345678
    memory_host_read_commands = 3456789
    memory_host_write_commands = 4567890
    memory_controller_busy_time = 5678
  }
  dimm_b2 = {
    size = "32GB"
    type = "DDR4-2933"
    manufacturer = "Samsung"
    part_number = "M393A4K40CB2-CVF"
    serial = "12345681"
    ecc = true
    rank = 2
    speed = 2933
    voltage = 1.2
    temperature = 35
    error_count = 0
    correctable_errors = 0
    uncorrectable_errors = 0
    memory_health = "good"
    memory_status = "ok"
    memory_temperature = 35
    memory_power_on_hours = 1234
    memory_power_cycles = 56
    memory_unsafe_shutdowns = 2
    memory_media_errors = 0
    memory_error_log_entries = 0
    memory_warning_temp_time = 0
    memory_critical_temp_time = 0
    memory_available_spare = 100
    memory_available_spare_threshold = 10
    memory_percentage_used = 5
    memory_data_units_read = 1234567
    memory_data_units_written = 2345678
    memory_host_read_commands = 3456789
    memory_host_write_commands = 4567890
    memory_controller_busy_time = 5678
  }
}

# CPU information
cpu_info = {
  socket_0 = {
    model = "Intel Xeon Gold 6230"
    cores = 20
    threads = 40
    base_clock = 2.1
    turbo_clock = 3.9
    cache = "27.5MB"
    tdp = 125
    temperature = 45
    power_consumption = 85
    cpu_health = "good"
    cpu_status = "ok"
    cpu_temperature = 45
    cpu_power_on_hours = 1234
    cpu_power_cycles = 56
    cpu_unsafe_shutdowns = 2
    cpu_media_errors = 0
    cpu_error_log_entries = 0
    cpu_warning_temp_time = 0
    cpu_critical_temp_time = 0
    cpu_available_spare = 100
    cpu_available_spare_threshold = 10
    cpu_percentage_used = 5
    cpu_data_units_read = 1234567
    cpu_data_units_written = 2345678
    cpu_host_read_commands = 3456789
    cpu_host_write_commands = 4567890
    cpu_controller_busy_time = 5678
  }
  socket_1 = {
    model = "Intel Xeon Gold 6230"
    cores = 20
    threads = 40
    base_clock = 2.1
    turbo_clock = 3.9
    cache = "27.5MB"
    tdp = 125
    temperature = 46
    power_consumption = 87
    cpu_health = "good"
    cpu_status = "ok"
    cpu_temperature = 46
    cpu_power_on_hours = 1234
    cpu_power_cycles = 56
    cpu_unsafe_shutdowns = 2
    cpu_media_errors = 0
    cpu_error_log_entries = 0
    cpu_warning_temp_time = 0
    cpu_critical_temp_time = 0
    cpu_available_spare = 100
    cpu_available_spare_threshold = 10
    cpu_percentage_used = 5
    cpu_data_units_read = 1234567
    cpu_data_units_written = 2345678
    cpu_host_read_commands = 3456789
    cpu_host_write_commands = 4567890
    cpu_controller_busy_time = 5678
  }
}

# Power supply information
power_supply_info = {
  psu_1 = {
    model = "PWS-1K60P-1R"
    wattage = 1600
    efficiency = "Platinum"
    temperature = 40
    power_consumption = 450
    psu_health = "good"
    psu_status = "ok"
    psu_temperature = 40
    psu_power_on_hours = 1234
    psu_power_cycles = 56
    psu_unsafe_shutdowns = 2
    psu_media_errors = 0
    psu_error_log_entries = 0
    psu_warning_temp_time = 0
    psu_critical_temp_time = 0
    psu_available_spare = 100
    psu_available_spare_threshold = 10
    psu_percentage_used = 5
    psu_data_units_read = 1234567
    psu_data_units_written = 2345678
    psu_host_read_commands = 3456789
    psu_host_write_commands = 4567890
    psu_controller_busy_time = 5678
  }
  psu_2 = {
    model = "PWS-1K60P-1R"
    wattage = 1600
    efficiency = "Platinum"
    temperature = 41
    power_consumption = 450
    psu_health = "good"
    psu_status = "ok"
    psu_temperature = 41
    psu_power_on_hours = 1234
    psu_power_cycles = 56
    psu_unsafe_shutdowns = 2
    psu_media_errors = 0
    psu_error_log_entries = 0
    psu_warning_temp_time = 0
    psu_critical_temp_time = 0
    psu_available_spare = 100
    psu_available_spare_threshold = 10
    psu_percentage_used = 5
    psu_data_units_read = 1234567
    psu_data_units_written = 2345678
    psu_host_read_commands = 3456789
    psu_host_write_commands = 4567890
    psu_controller_busy_time = 5678
  }
}

# Fan information
fan_info = {
  fan_1 = {
    speed = 1200
    temperature = 35
    fan_health = "good"
    fan_status = "ok"
    fan_temperature = 35
    fan_power_on_hours = 1234
    fan_power_cycles = 56
    fan_unsafe_shutdowns = 2
    fan_media_errors = 0
    fan_error_log_entries = 0
    fan_warning_temp_time = 0
    fan_critical_temp_time = 0
    fan_available_spare = 100
    fan_available_spare_threshold = 10
    fan_percentage_used = 5
    fan_data_units_read = 1234567
    fan_data_units_written = 2345678
    fan_host_read_commands = 3456789
    fan_host_write_commands = 4567890
    fan_controller_busy_time = 5678
  }
  fan_2 = {
    speed = 1200
    temperature = 36
    fan_health = "good"
    fan_status = "ok"
    fan_temperature = 36
    fan_power_on_hours = 1234
    fan_power_cycles = 56
    fan_unsafe_shutdowns = 2
    fan_media_errors = 0
    fan_error_log_entries = 0
    fan_warning_temp_time = 0
    fan_critical_temp_time = 0
    fan_available_spare = 100
    fan_available_spare_threshold = 10
    fan_percentage_used = 5
    fan_data_units_read = 1234567
    fan_data_units_written = 2345678
    fan_host_read_commands = 3456789
    fan_host_write_commands = 4567890
    fan_controller_busy_time = 5678
  }
  fan_3 = {
    speed = 1200
    temperature = 34
    fan_health = "good"
    fan_status = "ok"
    fan_temperature = 34
    fan_power_on_hours = 1234
    fan_power_cycles = 56
    fan_unsafe_shutdowns = 2
    fan_media_errors = 0
    fan_error_log_entries = 0
    fan_warning_temp_time = 0
    fan_critical_temp_time = 0
    fan_available_spare = 100
    fan_available_spare_threshold = 10
    fan_percentage_used = 5
    fan_data_units_read = 1234567
    fan_data_units_written = 2345678
    fan_host_read_commands = 3456789
    fan_host_write_commands = 4567890
    fan_controller_busy_time = 5678
  }
  fan_4 = {
    speed = 1200
    temperature = 35
    fan_health = "good"
    fan_status = "ok"
    fan_temperature = 35
    fan_power_on_hours = 1234
    fan_power_cycles = 56
    fan_unsafe_shutdowns = 2
    fan_media_errors = 0
    fan_error_log_entries = 0
    fan_warning_temp_time = 0
    fan_critical_temp_time = 0
    fan_available_spare = 100
    fan_available_spare_threshold = 10
    fan_percentage_used = 5
    fan_data_units_read = 1234567
    fan_data_units_written = 2345678
    fan_host_read_commands = 3456789
    fan_host_write_commands = 4567890
    fan_controller_busy_time = 5678
  }
}

# Temperature sensors
temperature_sensors = {
  cpu_0 = {
    temperature = 45
    sensor_health = "good"
    sensor_status = "ok"
    sensor_temperature = 45
    sensor_power_on_hours = 1234
    sensor_power_cycles = 56
    sensor_unsafe_shutdowns = 2
    sensor_media_errors = 0
    sensor_error_log_entries = 0
    sensor_warning_temp_time = 0
    sensor_critical_temp_time = 0
    sensor_available_spare = 100
    sensor_available_spare_threshold = 10
    sensor_percentage_used = 5
    sensor_data_units_read = 1234567
    sensor_data_units_written = 2345678
    sensor_host_read_commands = 3456789
    sensor_host_write_commands = 4567890
    sensor_controller_busy_time = 5678
  }
  cpu_1 = {
    temperature = 46
    sensor_health = "good"
    sensor_status = "ok"
    sensor_temperature = 46
    sensor_power_on_hours = 1234
    sensor_power_cycles = 56
    sensor_unsafe_shutdowns = 2
    sensor_media_errors = 0
    sensor_error_log_entries = 0
    sensor_warning_temp_time = 0
    sensor_critical_temp_time = 0
    sensor_available_spare = 100
    sensor_available_spare_threshold = 10
    sensor_percentage_used = 5
    sensor_data_units_read = 1234567
    sensor_data_units_written = 2345678
    sensor_host_read_commands = 3456789
    sensor_host_write_commands = 4567890
    sensor_controller_busy_time = 5678
  }
  system = {
    temperature = 35
    sensor_health = "good"
    sensor_status = "ok"
    sensor_temperature = 35
    sensor_power_on_hours = 1234
    sensor_power_cycles = 56
    sensor_unsafe_shutdowns = 2
    sensor_media_errors = 0
    sensor_error_log_entries = 0
    sensor_warning_temp_time = 0
    sensor_critical_temp_time = 0
    sensor_available_spare = 100
    sensor_available_spare_threshold = 10
    sensor_percentage_used = 5
    sensor_data_units_read = 1234567
    sensor_data_units_written = 2345678
    sensor_host_read_commands = 3456789
    sensor_host_write_commands = 4567890
    sensor_controller_busy_time = 5678
  }
}

# Voltage sensors
voltage_sensors = {
  cpu_0_vcore = {
    voltage = 1.8
    sensor_health = "good"
    sensor_status = "ok"
    sensor_temperature = 45
    sensor_power_on_hours = 1234
    sensor_power_cycles = 56
    sensor_unsafe_shutdowns = 2
    sensor_media_errors = 0
    sensor_error_log_entries = 0
    sensor_warning_temp_time = 0
    sensor_critical_temp_time = 0
    sensor_available_spare = 100
    sensor_available_spare_threshold = 10
    sensor_percentage_used = 5
    sensor_data_units_read = 1234567
    sensor_data_units_written = 2345678
    sensor_host_read_commands = 3456789
    sensor_host_write_commands = 4567890
    sensor_controller_busy_time = 5678
  }
  cpu_1_vcore = {
    voltage = 1.8
    sensor_health = "good"
    sensor_status = "ok"
    sensor_temperature = 46
    sensor_power_on_hours = 1234
    sensor_power_cycles = 56
    sensor_unsafe_shutdowns = 2
    sensor_media_errors = 0
    sensor_error_log_entries = 0
    sensor_warning_temp_time = 0
    sensor_critical_temp_time = 0
    sensor_available_spare = 100
    sensor_available_spare_threshold = 10
    sensor_percentage_used = 5
    sensor_data_units_read = 1234567
    sensor_data_units_written = 2345678
    sensor_host_read_commands = 3456789
    sensor_host_write_commands = 4567890
    sensor_controller_busy_time = 5678
  }
  system_12v = {
    voltage = 12.0
    sensor_health = "good"
    sensor_status = "ok"
    sensor_temperature = 35
    sensor_power_on_hours = 1234
    sensor_power_cycles = 56
    sensor_unsafe_shutdowns = 2
    sensor_media_errors = 0
    sensor_error_log_entries = 0
    sensor_warning_temp_time = 0
    sensor_critical_temp_time = 0
    sensor_available_spare = 100
    sensor_available_spare_threshold = 10
    sensor_percentage_used = 5
    sensor_data_units_read = 1234567
    sensor_data_units_written = 2345678
    sensor_host_read_commands = 3456789
    sensor_host_write_commands = 4567890
    sensor_controller_busy_time = 5678
  }
}

# Current sensors
current_sensors = {
  cpu_0_current = {
    current = 85
    sensor_health = "good"
    sensor_status = "ok"
    sensor_temperature = 45
    sensor_power_on_hours = 1234
    sensor_power_cycles = 56
    sensor_unsafe_shutdowns = 2
    sensor_media_errors = 0
    sensor_error_log_entries = 0
    sensor_warning_temp_time = 0
    sensor_critical_temp_time = 0
    sensor_available_spare = 100
    sensor_available_spare_threshold = 10
    sensor_percentage_used = 5
    sensor_data_units_read = 1234567
    sensor_data_units_written = 2345678
    sensor_host_read_commands = 3456789
    sensor_host_write_commands = 4567890
    sensor_controller_busy_time = 5678
  }
  cpu_1_current = {
    current = 87
    sensor_health = "good"
    sensor_status = "ok"
    sensor_temperature = 46
    sensor_power_on_hours = 1234
    sensor_power_cycles = 56
    sensor_unsafe_shutdowns = 2
    sensor_media_errors = 0
    sensor_error_log_entries = 0
    sensor_warning_temp_time = 0
    sensor_critical_temp_time = 0
    sensor_available_spare = 100
    sensor_available_spare_threshold = 10
    sensor_percentage_used = 5
    sensor_data_units_read = 1234567
    sensor_data_units_written = 2345678
    sensor_host_read_commands = 3456789
    sensor_host_write_commands = 4567890
    sensor_controller_busy_time = 5678
  }
}

# Power consumption
power_consumption = {
  total = 450
  cpu_0 = 85
  cpu_1 = 87
  memory = 120
  storage = 50
  network = 30
  fans = 40
  psu_1 = 225
  psu_2 = 225
}

# System health
system_health = {
  overall = "good"
  cpu = "good"
  memory = "good"
  storage = "good"
  network = "good"
  power = "good"
  fans = "good"
  temperatures = "good"
  voltages = "good"
  currents = "good"
}

# System status
system_status = {
  uptime = 1234
  load_average = [1.2, 1.1, 1.0]
  memory_usage = 45
  cpu_usage = 25
  disk_usage = 30
  network_usage = 15
  power_usage = 450
  temperature = 35
  fan_speed = 1200
  voltage = 12.0
  current = 85
}

# System information
system_info = {
  hostname = "supermicro-node01"
  domain = "datacenter.local"
  os = "ubuntu-22.04"
  kernel = "5.15.0-91-generic"
  uptime = 1234
  load_average = [1.2, 1.1, 1.0]
  memory_usage = 45
  cpu_usage = 25
  disk_usage = 30
  network_usage = 15
  power_usage = 450
  temperature = 35
  fan_speed = 1200
  voltage = 12.0
  current = 85
}

# System configuration
system_config = {
  hostname = "supermicro-node01"
  domain = "datacenter.local"
  os = "ubuntu-22.04"
  kernel = "5.15.0-91-generic"
  uptime = 1234
  load_average = [1.2, 1.1, 1.0]
  memory_usage = 45
  cpu_usage = 25
  disk_usage = 30
  network_usage = 15
  power_usage = 450
  temperature = 35
  fan_speed = 1200
  voltage = 12.0
  current = 85
}
```

```
# ansible_managed = {{ ansible_managed }}
# Generated by Ansible
# Do not edit manually

# Global settings
global
    log /dev/log local0 info
    log /dev/log local1 notice
    chroot /var/lib/haproxy
    stats socket /run/haproxy/admin.sock mode 660 level admin
    stats timeout 30s
    user haproxy
    group haproxy
    daemon
    maxconn 4096
    tune.ssl.default-dh-param 2048
    ssl-default-bind-ciphers ECDHE-ECDSA-AES128-GCM-SHA256:ECDHE-RSA-AES128-GCM-SHA256:ECDHE-ECDSA-AES256-GCM-SHA384:ECDHE-RSA-AES256-GCM-SHA384:ECDHE-ECDSA-CHACHA20-POLY1305:ECDHE-RSA-CHACHA20-POLY1305:DHE-RSA-AES128-GCM-SHA256:DHE-RSA-AES256-GCM-SHA384
    ssl-default-bind-options ssl-min-ver TLSv1.2 no-tls-tickets

# Default settings
defaults
    log global
    mode http
    option httplog
    option dontlognull
    option forwardfor
    option http-server-close
    timeout connect 5000
    timeout client 50000
    timeout server 50000
    errorfile 400 /etc/haproxy/errors/400.http
    errorfile 403 /etc/haproxy/errors/403.http
    errorfile 408 /etc/haproxy/errors/408.http
    errorfile 500 /etc/haproxy/errors/500.http
    errorfile 502 /etc/haproxy/errors/502.http
    errorfile 503 /etc/haproxy/errors/503.http
    errorfile 504 /etc/haproxy/errors/504.http

# Stats page
listen stats
    bind *:8404
    stats enable
    stats uri /stats
    stats refresh 10s
    stats admin if LOCALHOST
    stats auth admin:{{ haproxy_stats_password }}

# Frontend for HTTP traffic
frontend http_front
    bind *:80
    redirect scheme https code 301 if !{ ssl_fc }

# Frontend for HTTPS traffic
frontend https_front
    bind *:443 ssl crt /etc/ssl/certs/haproxy.pem
    http-request set-header X-Forwarded-Proto https
    http-request set-header X-Forwarded-Port 443
    http-request add-header X-Request-Start %Ts.%ms
    default_backend app_servers

# Backend application servers
backend app_servers
    balance roundrobin
    option httpchk GET /health HTTP/1.1\r\nHost:\ localhost
    http-check expect status 200
    server app1 10.0.1.10:8080 check inter 2000 rise 2 fall 3 maxconn 1000
    server app2 10.0.1.11:8080 check inter 2000 rise 2 fall 3 maxconn 1000
    server app3 10.0.1.12:8080 check inter 2000 rise 2 fall 3 maxconn 1000 backup

# Backend for API servers with sticky sessions
backend api_servers
    balance leastconn
    cookie SERVERID insert indirect nocache
    option httpchk GET /api/v1/health HTTP/1.1\r\nHost:\ localhost
    http-check expect status 200
    server api1 10.0.2.10:3000 check cookie api1 inter 2000 rise 2 fall 3 maxconn 500
    server api2 10.0.2.11:3000 check cookie api2 inter 2000 rise 2 fall 3 maxconn 500
    server api3 10.0.2.12:3000 check cookie api3 inter 2000 rise 2 fall 3 maxconn 500 backup

# Backend for static assets
backend static_servers
    balance roundrobin
    server static1 10.0.3.10:80 check inter 5000 rise 2 fall 3
    server static2 10.0.3.11:80 check inter 5000 rise 2 fall 3 backup

# Rate limiting
frontend rate_limited
    bind *:8443 ssl crt /etc/ssl/certs/haproxy.pem
    stick-table type ip size 100k expire 30s store http_req_rate(10s)
    http-request track-sc0 src
    http-request deny deny_status 429 if { sc_http_req_rate(0) gt 100 }
    default_backend app_servers

# Maintenance page
frontend maintenance
    bind *:9000
    errorfile 503 /etc/haproxy/errors/maintenance.http
    default_backend maintenance_backend

backend maintenance_backend
    errorfile 503 /etc/haproxy/errors/maintenance.http
```

```
# Application Configuration
# Environment: {{ environment }}

[server]
host = 0.0.0.0
port = {{ app_port }}
workers = {{ ansible_processor_vcpus * 2 }}
timeout = 30
keepalive = 65

[database]
driver = postgresql
host = {{ db_host }}
port = {{ db_port }}
name = {{ db_name }}
user = {{ db_user }}
password = {{ db_password }}
pool_size = 20
max_overflow = 10
pool_timeout = 30
pool_recycle = 1800

[redis]
host = {{ redis_host }}
port = {{ redis_port }}
db = 0
password = {{ redis_password }}
socket_timeout = 5
connection_pool_size = 50

[cache]
backend = redis
default_ttl = 300
session_ttl = 86400
max_entries = 10000

[logging]
level = {{ log_level | default('info') }}
format = %(asctime)s %(levelname)s %(name)s %(message)s
file = /var/log/app/application.log
max_bytes = 10485760
backup_count = 5
rotate = true

[security]
secret_key = {{ secret_key }}
jwt_algorithm = HS256
jwt_expiry = 3600
refresh_token_expiry = 604800
password_min_length = 12
password_require_special = true
max_login_attempts = 5
lockout_duration = 900
cors_origins = {{ cors_origins | join(',') }}
csrf_enabled = true

[email]
smtp_host = {{ smtp_host }}
smtp_port = {{ smtp_port }}
smtp_user = {{ smtp_user }}
smtp_password = {{ smtp_password }}
smtp_tls = true
from_address = noreply@{{ domain }}
from_name = {{ app_name }}

[storage]
backend = s3
bucket = {{ s3_bucket }}
region = {{ s3_region }}
access_key = {{ s3_access_key }}
secret_key = {{ s3_secret_key }}
max_upload_size = 52428800
allowed_extensions = jpg,jpeg,png,gif,pdf,doc,docx,xls,xlsx,csv

[monitoring]
enabled = true
metrics_port = 9090
health_check_path = /health
sentry_dsn = {{ sentry_dsn }}
sentry_environment = {{ environment }}
sentry_traces_sample_rate = 0.1

[feature_flags]
new_dashboard = {{ feature_new_dashboard | default('false') }}
beta_api = {{ feature_beta_api | default('false') }}
maintenance_mode = {{ maintenance_mode | default('false') }}
rate_limiting = true
two_factor_auth = {{ feature_2fa | default('false') }}

[notifications]
slack_webhook = {{ slack_webhook_url }}
slack_channel = #{{ slack_channel | default('alerts') }}
pagerduty_key = {{ pagerduty_integration_key }}
notify_on_error = true
notify_on_deploy = true
```

```
```

```
# Configuration file for the Sphinx documentation builder.
#
# This file only contains a selection of the most common options. For a full
# list see the documentation:
# https://www.sphinx-doc.org/en/master/usage/configuration.html

# -- Path setup --------------------------------------------------------------

# If extensions (or modules to document with autodoc) are in another directory,
# add these directories to sys.path here. If the directory is relative to the
# documentation root, use os.path.abspath to make it absolute, like shown here.
#
import os
import sys
sys.path.insert(0, os.path.abspath('../..'))


# -- Project information -----------------------------------------------------

project = 'Python Project Documentation'
copyright = '2019, Read the Docs'
author = 'Read the Docs'

# The short X.Y version
version = ''
# The full version, including alpha/beta/rc tags
release = '0.1'


# -- General configuration ---------------------------------------------------

# If your documentation needs a minimal Sphinx version, state it here.
#
needs_sphinx = '1.0'

# Add any Sphinx extension module names here, as strings. They can be
# extensions coming with Sphinx (named 'sphinx.ext.*') or your custom
# ones.
extensions = [
    'sphinx.ext.autodoc',
    'sphinx.ext.doctest',
    'sphinx.ext.todo',
    'sphinx.ext.coverage',
    'sphinx.ext.mathjax',
    'sphinx.ext.ifconfig',
    'sphinx.ext.viewcode',
    'sphinx.ext.githubpages',
]

# Add any paths that contain templates here, relative to this directory.
templates_path = ['_templates']

# The suffix(es) of source filenames.
# You can specify multiple suffix as a list of string:
#
# source_suffix = ['.rst', '.md']
source_suffix = '.rst'

# The master toctree document.
master_doc = 'index'

# The language for content autogenerated by Sphinx. Refer to documentation
# for a list of supported languages.
#
# This is also used if you do content translation via gettext catalogs.
# Usually you set "language" from the command line for these cases.
language = None

# List of patterns, relative to source directory, that match files and
# directories to ignore when looking for source files.
# This pattern also affects html_static_path and html_extra_path .
exclude_patterns = [u'_build', 'Thumbs.db', '.DS_Store']

# The name of the Pygments (syntax highlighting) style to use.
pygments_style = 'sphinx'

# The default sidebars (for documents that don't match any pattern) for
# HTML output.  Overrides html_sidebars.  The path may contain
# glob-style patterns.
#
# This also supports the special template entry "**" which matches
# the template used for all existing documents that are not
# explicitly named.  Example:
#
# html_sidebars = {
#    '**': ['localtoc.html', 'relations.html'],
#    'index': ['globaltoc.html'],
# }


# -- Options for HTML output -------------------------------------------------

# The theme to use for HTML and HTML Help pages.  See the documentation for
# a list of builtin themes.
#
html_theme = 'alabaster'

# Theme options are theme-specific and customize the look and feel of a theme
# further.  For a list of options available for each theme, see the
# documentation.
#
# html_theme_options = {}

# Add any paths that contain custom static files (such as style sheets) here,
# relative to this directory. They are copied after the builtin static files,
# so a file named "default.css" will overwrite the builtin "default.css".
html_static_path = ['_static']

# Custom sidebar templates, must be a dictionary that maps document names
# to template names.
#
# The default sidebars (for documents that don't match any pattern) are
# defined by theme itself.  Builtin themes are using these templates by
# default: ``['localtoc.html', 'relations.html', 'sourcelink.html',
# 'searchbox.html']``.
#
# html_sidebars = {}


# -- Options for HTMLHelp output ---------------------------------------------

# Output file base name for HTML help builder.
htmlhelp_basename = 'PythonProjectDocumentationdoc'


# -- Options for LaTeX output ------------------------------------------------

latex_elements = {
    # The paper size ('letterpaper' or 'a4paper').
    #
    # 'papersize': 'letterpaper',

    # The font size ('10pt', '11pt' or '12pt').
    #
    # 'pointsize': '10pt',

    # Additional stuff for the LaTeX preamble.
    #
    # 'preamble': '',

    # Latex figure (float) alignment
    #
    # 'figure_align': 'htbp',
}

# Grouping the document tree into LaTeX files. List of tuples
# (source start file, target name, title,
#  author, documentclass [howto, manual, or own class]).
latex_documents = [
    (master_doc, 'PythonProjectDocumentation.tex', u'Python Project Documentation Documentation',
     u'Read the Docs', 'manual'),
]


# -- Options for manual page output ------------------------------------------

# One entry per manual page. List of tuples
# (source start file, name, description, authors, manual section).
man_pages = [
    (master_doc, 'pythonprojectdocumentation', u'Python Project Documentation Documentation',
     [author], 1)
]


# -- Options for Texinfo output ----------------------------------------------

# Grouping the document tree into Texinfo files. List of tuples
# (source start file, target name, title, author,
#  dir menu entry, description, category)
texinfo_documents = [
    (master_doc, 'PythonProjectDocumentation', u'Python Project Documentation Documentation',
     author, 'PythonProjectDocumentation', 'One line description of project.',
     'Miscellaneous'),
]


# -- Options for Epub output -------------------------------------------------

# Bibliographic Dublin Core info.
epub_title = project

# The unique identifier of the text. This can be a ISBN number
# or the project homepage.
#
# epub_identifier = ''

# A unique identification for the text.
#
# epub_uid = ''

# A list of files that should not be packed into the epub file.
epub_exclude_files = ['search.html']


# -- Extension configuration -------------------------------------------------

# -- Options for todo extension ----------------------------------------------

# If true, `todo` and `todoList` produce output, else they produce nothing.
todo_include_todos = True
```